Pico BASIC User's Manual

Pico BASIC for the RP2350 · October 31, 2026

Contents

1. Getting started
   - Boards
   - Loading it onto the board
   - Connecting
   - The terminal
   - First program
   - Programs without line numbers
   - Formatted listings
   - AUTORUN.BAS
   - Editing a line
   - Full-screen editor
2. Program and direct commands
3. Language
   - Numbers, strings and variables
   - Arrays
   - Operators, highest priority first
   - Flow of control
   - Variables and data
   - Text output and input
   - PRINT USING
   - Functions
   - Error trapping
   - Events
   - Memory: PEEK and POKE
4. The flash disk and data files
   - The disk
   - The SD card (drive SD:)
   - Settings: CONFIG.TXT
   - Files from programs
5. Moving files to and from the PC
   - Pasting and listing
   - XMODEM: XRECV and XSEND
6. Sound
7. Hardware: GPIO, PWM, ADC, I2C, SPI, COM1
   - Pins
   - I2C and SPI
   - NeoPixels: WS2812B LEDs
   - Serial port (COM1)
   - The Cytron Maker Pi Pico
8. Memory
9. Errors
10. Limits
11. Example programs
12. Command reference
    - Index by topic
    - + - * / ^
    - = <> < > <= >=
    - ?
    - ABS
    - ADC
    - AND
    - ASC
    - ATN
    - AUTO
    - BEEP
    - BLOAD
    - BSAVE
    - BYE
    - CDBL
    - CHAIN
    - CHDIR
    - CHR$
    - CINT
    - CLEAR
    - CLOSE
    - CLS
    - COLOR
    - COM(1)
    - COMMON
    - CONFIG
    - CONT
    - COS
    - CSNG
    - CSRLIN
    - CVD
    - CVI
    - CVS
    - DATA
    - DATE$
    - DEF FN
    - DEF SEG
    - DEFDBL
    - DEFINT
    - DEFSNG
    - DEFSTR
    - DELETE
    - DIM
    - DIR$
    - EDIT
    - END
    - ENVIRON
    - ENVIRON$
    - EOF
    - EQV
    - ERASE
    - ERDEV
    - ERL
    - ERR
    - ERROR
    - EXIT FOR
    - EXP

EXTERR
FIELD
FILES
FIX
FOR
FORMAT
FRE
FREEFILE
GET
GOSUB
GOTO
HEX$
I2C
I2CREAD
I2CREAD$
IF
IMP
INKEY$
INPUT
INPUT
INPUT$
INSTR
INT
KEY
KEY(n)
KILL
LCASE$
LEFT$
LEN
LET
LINE INPUT
LINE INPUT
LIST
LLIST
LOAD
LOC
LOCATE
LOF
LOG
LPRINT
LSET
MERGE
MID$
MID$ =
MKD$
MKDIR
MKI$
MKS$
NAME
NEOPIXEL
NEW
NEXT
NOT
NUMBERS
OCT$
ON COM
ON ERROR GOTO
ON KEY
ON PLAY
ON TIMER
ON...GOSUB
ON...GOTO
OPEN
OPTION BASE
OR
PAUSE
PEEK
PIN
PINMODE
PINS
PLAY
PLAY ON
PLAY(n)
POKE
POS
PRINT
PRINT
PRINT USING
PUT
PWM
RANDOMIZE
READ
REM
RENUM
RESET
RESTORE
RESUME
RETURN
RIGHT$
RMDIR
RND
RSET
RUN
SAVE
SCREEN(r, c)
SERVO
SGN
SIN
SOUND
SPACE$
SPC
SPI
SPI$
SQR
STOP
STR$
STRING$
SWAP
SYSTEM
TAB
TAN
TEMP
TIME$
TIMER
TIMER ON
TROFF
TRON
UCASE$
VAL
VIEW PRINT
WEND
WHILE
WIDTH
WRITE
WRITE
XOR
XRECV
XSEND
\ and MOD
Appendix A: error codes
Appendix B: PRINT USING formats
Appendix C: INKEY$ key codes
Appendix D: characters
Appendix E: pins BASIC uses

Pico BASIC is a GW-BASIC style interpreter for a bare RP2350 or RP2040 board, used from a serial terminal. It keeps its programs and data files on a FAT disk in the board's own flash, swaps files with the PC by XMODEM, plays tones on a speaker pin, and drives the chip's GPIO, PWM, ADC, I2C, SPI and a second UART. It needs no HDMI, no keyboard and no USB drive: just the board and a USB cable to the PC, where it appears as a serial port.

It is the BASIC of the Pico 2 mini OS, built on its own for a serial terminal.

1. Getting started

Boards

Board	Chip	Flash disk	Memory for BASIC
Raspberry Pi Pico 2	RP2350A, GPIO 0-29	about 3 MB	about 450 KB (SRAM)
Pimoroni Pico Plus 2	RP2350B, GPIO 0-47	about 15 MB	about 8.4 MB (8 MB PSRAM + SRAM)
Raspberry Pi Pico (the original)	RP2040, GPIO 0-29	about 1 MB	about 190 KB (SRAM)

Each board needs its own build: `build/pico_basic.uf2` for the Pico 2, `build-plus2/pico_basic.uf2` for the Pico Plus 2 and `build-pico/pico_basic.uf2` for the Pico (see the README). On the RP2040, BASIC is the same language, but with less memory and no floating-point hardware, so arithmetic-heavy programs run several times slower than on an RP2350, and a 32 KB stack there has no hardware guard (BASIC still checks it itself).

Loading it onto the board

- **BOOTSEL:** hold the BOOTSEL button while plugging the board into USB, then copy the `.uf2` file to the drive that appears. The board restarts into BASIC, and the same USB port becomes its console.
- **picotool:** while BASIC runs, `picotool load -f build/pico_basic.uf2` (or `picotool reboot -f -u`, to get the BOOTSEL drive) reboots it over its USB console port: no button needed.
- **Debug Probe (SWD):** with Raspberry Pi's OpenOCD:

```
OCD=$(ls -d ~/.pico-sdk/openocd/*/ | tail -1)
"$OCD/openocd" -s "$OCD/scripts" -f interface/cmsis-dap.cfg -f target/rp2350.cfg \
    -c "adapter speed 5000" -c "program build-plus2/pico_basic.elf verify reset
exit"
```

Loading a new BASIC keeps the files on the flash disk (it lives above the program). Loading some other program that is bigger than 1 MB, or that uses the flash for itself, overwrites it.

Connecting

The console is the board's own **USB port**: connect it to the PC with a data USB cable (a charge-only cable or a power supply won't do). It appears as a USB serial port: `/dev/ttyACM0` (or the next free number, `ttyACM1` when a Debug Probe is plugged in too) on Linux, a `COM` port on Windows, `/dev/cu.usbmodem...` on a Mac. On Linux, `ls -l /dev/serial/by-id/` shows it as `usb-Raspberry_Pi_Pico_....`.

The only other wiring is optional:

Board pin	Connect to
GP18 (pin 24)	a piezo buzzer or small speaker (through about 100 Ω) to GND, for BEEP, SOUND and PLAY (the Maker Pi Pico's own buzzer)
GP10-15, 3V3, GND	a microSD card slot, as drive `SD:` (section 4; the Maker Pi Pico has one; other pins: CONFIG.TXT)
SWD (the debug pins)	a Raspberry Pi Debug Probe, for loading programs with OpenOCD (its UART isn't used)

3.3 V levels only on every pin: never connect RS-232 or 5 V signals straight to them.

The terminal

Any terminal program: the speed setting doesn't matter on a USB serial port (115200 8N1 is fine). Use ANSI or VT100, with the **UTF-8** character set, and make the window at least **80 columns by 30 rows** (BASIC's screen; LOCATE takes rows 1-30). For example:

```
picocom /dev/ttyACM1
minicom -D /dev/ttyACM1
```

On Windows, PuTTY or Tera Term on the board's COM port. gtkterm and screen work too, but can't send files by XMODEM (section 5). On Linux your user needs to be in the `dialout` group to open the port.

When the board starts it shows:

```
Pico BASIC 0.1 for the RP2350 (pimoroni_pico_plus2_rp2350)
PSRAM: 8 MB
Flash disk: 15336 KB, 15332 KB free
Pico BASIC (GW-BASIC style). SYSTEM starts it afresh.
8825424 Bytes free
Ok
```

BASIC waits up to 3 seconds at start-up for a terminal; a terminal that connects later (or reconnects) gets the first three lines again and an `Ok`. The very first start-up formats the flash disk (a second or two).

First program

Lines that start with a number are stored as the program; anything else runs straight away:

```
10 FOR I = 1 TO 3
20   PRINT "HELLO"; I
30 NEXT
RUN
SAVE "HELLO"
```

- Typing a line number alone deletes that line; retyping one replaces it.
- **Ctrl-C** stops a running program with `Break in <line>`; `CONT` carries on from there.
- `SYSTEM` (or `BYE`) clears everything and starts BASIC afresh, as after a reset (files on the disk stay).

Programs without line numbers

Line numbers are optional. A program written in the full-screen editor (`EDIT`, below), or in a text file on the PC, can leave them out and use **labels** instead: a name and a colon starting a line. GOTO, GOSUB, ON ... GOTO/GOSUB, IF ... THEN/ELSE, RESTORE, RESUME, ON ERROR GOTO, ON KEY(n) GOSUB and RUN take a label wherever they take a line number:

```
' Count to 3, then stop
I = 0
again:
  I = I + 1
  PRINT "HELLO"; I
  IF I < 3 THEN again
GOSUB done
END

done: PRINT "Done"
RETURN
```

- A label is letters, digits, `.` and `_`, starting with a letter, up to 40 characters; any case (BASIC shows it in capitals, as it does names). It can have statements after it on its line (`done: PRINT "Done"`). Two of the same name stop RUN with `Duplicate label`; a jump to one that isn't there is `Undefined label`.
- A statement keyword alone or with a number after it is a statement, not a label: `CLS:` and `SCREEN0:` are statements, `ENDGAME:` is a label.
- After THEN or ELSE a label jumps when nothing else follows it (`THEN again`); `THEN GOTO again` always works.
- A file's lines all have numbers or none do (LOAD says `Bad file mode` if it mixes them). Without numbers, each line's number is its line in the file, blank lines too: errors say `Syntax error in line 12` (Ctrl-G 12 in the editor goes there), and ERL and TRON give these numbers. LIST, SAVE and EDIT show the program without them.
- Labels work in numbered programs too.
- Typing a numbered line, AUTO, `EDIT n`, RENUM and MERGE need line numbers; with a program without them, use `EDIT`.
- `NUMBERS OFF` takes the numbers off a program: each line something jumps to gets a label `L` and its old number (`L100:`), and the jumps use it. `NUMBERS ON` numbers the lines 10, 20, 30... (labels stay). Numbers in `ERL` comparisons (`IF ERL = 100`) aren't changed.
- `examples/LABELS.BAS` is a menu program with labels.

Formatted listings

LIST, EDIT and SAVE show the program indented: the bodies of FOR ... NEXT and WHILE ... WEND loops go in two spaces a level, and in a program without line numbers so do the subroutines. Labels stay at the left; blank lines are kept:

```
' Count to 3, then stop
I = 0
again:
I = I + 1
FOR J = 1 TO I
  PRINT J;
NEXT
IF I < 3 THEN again
GOSUB done
END

done:
  PRINT "Done"
  RETURN
```

- A subroutine is a label that GOSUB, ON ... GOSUB or ON KEY(n) GOSUB calls, or ON ERROR GOTO goes to. Its lines are indented up to the next subroutine, or the next label after the RETURN or RESUME that ends it. Other labels (GOTO's, RESTORE's) don't change the indent.
- Only the look changes: loading a program (and leaving EDIT) takes the indents off again, so you can indent as you like or not at all.
- In a numbered program the indent goes after the number (`20 PRINT J;`).
- `FORMAT OFF` shows the lines as stored, `FORMAT ON` indents again, and `FORMAT n` sets the spaces a level (1-8).
- NEXT and WEND that close more blocks than are open just stop at the left.

AUTORUN.BAS

At power-up (and after a reset, but not after `SYSTEM`) BASIC runs `AUTORUN.BAS` if there is one: on the SD card first, so a card can override, else on the flash disk. BASIC waits up to 3 seconds for a terminal to connect first, so a terminal that is open sees everything the program prints; a board on its own (on a USB charger, say) runs it after those 3 seconds. What it prints while no terminal is connected is kept (the last 8 KB, 4 KB on the RP2040) and shown when one connects, after the banner and Bytes free, between "--- printed while no terminal was connected ---" and "---", then "A program is running" (or Ok). When it ends, or stops with an error or Ctrl-C, the `Ok` prompt follows as usual.

```
NEW
10 NEOPIXEL OPEN: NEOPIXEL BRIGHT 40
20 NEOPIXEL FILL 0, 0, 255: NEOPIXEL SHOW
SAVE "AUTORUN"
```

To start without it: hold **GP20** low at power-up (the Maker Pi Pico's first button), or press **Esc** or **Ctrl-C** within a second if the terminal is already open (the banner says so). `KILL "AUTORUN.BAS"` removes it. `RUN "NEODEMO"` saved as AUTORUN (`LOAD "NEODEMO"` then `SAVE "AUTORUN"`) makes the board start into the demo.

Editing a line

At the `Ok` prompt and in `INPUT`:

Key	Does
Left, Right, Home, End	move along the line
Backspace, Delete	delete before / at the cursor
Insert	switches between inserting and overwriting
Esc	clears the line
Up, Down	(at `Ok`) bring back the last 8 lines typed
F1-F10	type their soft-key text (`KEY LIST` shows it: F1 `LIST`, F2 `RUN` and Enter, ... as in GW-BASIC)
Ctrl-C	abandons the line

`EDIT n` (or `EDIT .`, the last line typed or the one that failed) puts a program line on the input line to change; Enter stores it. `EDIT` alone opens the whole program in the full-screen editor (below). A long line runs on over several rows. Pasting a whole listing into the terminal at the `Ok` prompt enters it line by line.

Full-screen editor

`EDIT` shows the program one line per row, to move around and change freely; `EDIT "file"` edits any text file (on `FLASH:` or `SD:`) the same way. It asks the terminal its size (80 x 24 if it doesn't say), and the bottom line shows where you are: `Program * Ln 3/12 Col 5 INS`.

Key	Does
Arrows, Home, End, PgUp, PgDn	move
Ctrl-Home, Ctrl-End	the top, the bottom
Backspace, Delete	delete (at a line's ends they join lines)
Enter	split the line
Insert	insert or overwrite
Tab	spaces to the next column of 4
Shift+Up, Shift+Down	mark lines (shown in reverse)
Ctrl-X	cut the marked lines, or the cursor's line (pressed again, the next line is added)
Ctrl-C	copy them (the cursor moves down, so pressing again adds the next line)
Ctrl-V	paste above the cursor's line
Ctrl-Z, Ctrl-R	undo, redo (typing or deleting in a row is one step; up to 100 steps)
Ctrl-F, F3	find, find the next (any case)
Ctrl-G	go to a line (in a program: a BASIC line number)
Ctrl-S	save
Esc or Ctrl-Q	leave (asks whether to save the changes)
Ctrl-L	draw the screen again
F1	the keys

When a program is saved, every line that isn't blank must start with a line number (0-65529, 255 characters at most); otherwise nothing is saved, the cursor goes to the line, and the bottom line says why. The lines are then taken as if typed: in any order, blank ones left out. Leaving without saving keeps the program as it was. Ctrl-S needs the terminal's flow control off (XON/XOFF would swallow it): gtkterm, minicom and picocom have it off unless set. In the editor Ctrl-C copies; it stops programs only at the Ok prompt and while they run. gtkterm keeps Ctrl-C and Ctrl-V for the editor (its own copy and paste are Shift+Ctrl+C and Shift+Ctrl+V).

2. Program and direct commands

Command	What it does
`RUN`	Runs the program from the start (clears variables first).
`RUN line`	Runs from that line (or label).
`RUN "file"`	Loads `file` and runs it.
`LIST [range]`	Lists the program, pausing every 28 lines (any key goes on, Q stops). A range is `n`, `n-`, `-n` or `n-m`.
`LLIST [range]`	Same as LIST (there's no printer).
`NEW`	Deletes the program and its variables.
`CLEAR`	Clears variables and closes files (any arguments are ignored).
`LOAD "file"[,R]`	Loads a program, and with `,R` runs it. `.BAS` is added if the name has no extension. Reads plain text and GW-BASIC tokenised files.
`SAVE "file"[,A]`	Saves as plain text (`,A` is accepted; text is the only format). `.BAS` is added if the name has no extension.
`MERGE "file"`	Adds the lines of a plain-text program to the one in memory (lines with the same number are replaced).
`CHAIN [MERGE] "file"[, [line][, ALL][, DELETE range]]`	Runs another program from a program, keeping open files. Only the variables named in `COMMON` are passed on, or all of them with `ALL`. `CHAIN MERGE` adds the file's lines (after deleting `range`) and keeps all variables.
`COMMON var[, var...]`	Names the variables CHAIN passes on (arrays as `A()`).
`FILES ["pattern"]`	Lists the files in the current folder (`FILES "*.BAS"`), then the disk's free space: `15331328 Bytes free`.
`KILL "file"`	Deletes a file.
`NAME "old" AS "new"`	Renames a file.
`DELETE range`	Deletes program lines.
`RENUM [new][,[old][,step]]`	Renumbers lines (default 10, first, 10), and the GOTO/GOSUB/THEN/ELSE/RESTORE/RESUME/RUN targets with them.
`FORMAT ON` / `FORMAT OFF` / `FORMAT n`	LIST, EDIT and SAVE indent loop bodies and the lines under labels (on at start-up; `n`: spaces a level, 1-8).
`NUMBERS OFF` / `NUMBERS ON`	Takes a program's line numbers off (the lines jumped to get labels such as `L100:`), or numbers it 10, 20, 30... (section 1).
`AUTO [start][,step]`	Numbers new lines for you (default 10,10). A `*` after the number means that line exists: Enter keeps it. An empty line where none exists, or Ctrl-C, stops.
`EDIT n` / `EDIT .`	Puts line `n` on the input line to edit.
`CONT`	Continues after Ctrl-C or STOP.
`TRON` / `TROFF`	Turns line-number tracing on/off: each line's number is printed in brackets as it runs.
`SYSTEM` / `BYE`	Starts BASIC afresh.

Example:

```
LOAD "PRIMES": LIST 10-50
RENUM 100, , 10
SAVE "PRIMES2"
```

3. Language

Numbers, strings and variables

- **Single precision** (`!`, and names without a type) keep 7 digits, as in GW-BASIC: a value stored in one is rounded to single precision and prints with up to 7 digits.
- **Double precision** (`#`) keep about 15 digits (printed with up to 9); calculations are always done in double precision.
- **Integers** (`%`) hold whole numbers from -32768 to 32767: a value stored in one is rounded (2.5 becomes 3), and one out of range gives Overflow.
- **Constants:** `1.5`, `1E6`, `3#`, `5%`, hex `&HFF`, octal `&O17`.
- **Strings** (`$`) are up to 255 characters and may hold any byte, including CHR$(0).
- **Names** are up to 40 characters: letters, digits and `.`, starting with a letter, with an optional type character. `A$`, `A%`, `A!` and `A#` are four different variables, and `A` is `A!` unless `DEFINT`/`DEFSNG`/ `DEFDBL`/`DEFSTR` say otherwise (after `DEFINT A-Z`, `A` is `A%`).
- An array and a plain variable may share a name: `A` and `A(3)` are separate.
- Keywords are recognised without spaces (GW-BASIC style), so a variable name must not start with a keyword: `PRINTER` reads as `PRINT ER`.

Arrays

- `DIM A(20), B$(5,5)`: indices start at 0; up to 8 dimensions.
- An array used without DIM gets 0-10 in each dimension.
- `OPTION BASE 1` (before any array is used) makes subscripts start at 1.
- `ERASE A, B$` deletes arrays, so they can be dimensioned again.

Operators, highest priority first

Operators	Meaning
`^`	power
`-`	negation
`*` `/`	multiply, divide
`\`	integer division
`MOD`	remainder
`+` `-`	add, subtract (`+` also joins strings)
`=` `<>` `<` `>` `<=` `>=`	comparisons, giving -1 for true and 0 for false (strings compare too)
`NOT`	
`AND`	
`OR` `XOR`	
`EQV` `IMP`	

The logical operators work bit by bit on integers, as in GW-BASIC.

Flow of control

Statement	Notes
`GOTO line`	`GOTO 100`, or a label: `GOTO again` (section 1)
`GOSUB line` … `RETURN`	RETURN ends the FOR and WHILE loops the subroutine started.
`ON n GOTO l1, l2, ...` / `ON n GOSUB ...`	Goes to the n-th line; past the end, carries on.
`IF cond THEN statements [ELSE statements]`	`THEN 100` and `ELSE 200` jump to lines; `IF X GOTO 100` works too. `IF A > 0 THEN PRINT "+" ELSE PRINT "-"`
`FOR v = a TO b [STEP s]` … `NEXT [v[, v...]]`	`FOR I = 10 TO 1 STEP -1: PRINT I: NEXT`
`EXIT FOR`	Leaves the innermost FOR loop (from QuickBASIC).
`WHILE cond` … `WEND`	`WHILE INKEY$ = "": WEND` waits for a key.
`END`	Ends the program (closes files).
`STOP`	Stops with `Break in <line>`; CONT goes on.
`PAUSE ms`	Waits (an extension); Ctrl-C still works. `PAUSE 500`
`REM text` or `' text`	A remark.

Several statements can share a line, separated by `:`.

Variables and data

Statement	Notes
`LET var = expr`	`LET` is optional: `X = 5`.
`DIM`, `ERASE`, `OPTION BASE`	See Arrays.
`DEFINT`, `DEFSNG`, `DEFDBL`, `DEFSTR letters`	`DEFINT A-Z`, `DEFSTR S, T`. RUN, NEW and CLEAR reset them.
`DATA items`, `READ var[, var...]`, `RESTORE [line]`	`10 DATA 3, "RED", 2.5` / `20 READ N, C$, X`
`SWAP a, b`	Exchanges two variables of the same type.
`DEF FNname[(args)] = expr`	`DEF FNSQ(X) = X * X`, then `PRINT FNSQ(3)`.
`MID$(v$, start[, n]) = x$`	Replaces characters of `v$` in place (its length never changes).
`RANDOMIZE [seed]`	Without a seed it uses the clock.
`DATE$ = "mm-dd-yyyy"`, `TIME$ = "hh:mm:ss"`	Sets the clock (`-` or `/` and two-digit years are accepted).
`ENVIRON "NAME=value"`	Sets a string `ENVIRON$` returns (there's no DOS environment).

The board has no battery-backed clock: at power-up the clock starts at 01-01-1970 00:00:00 and counts from there, until `DATE$` and `TIME$` set it. Times are UTC. Files saved before it is set are dated 1 January 2026.

Text output and input

Statement	Notes
`PRINT items` or `? items`	`;` keeps the cursor on the line, `,` jumps to the next 14-column zone; `TAB(n)` and `SPC(n)`. `PRINT "X ="; X, "Y ="; Y`
`PRINT USING fmt$; values`	Formatted numbers and strings (below). `PRINT USING "##.##"; 3.14159`
`LPRINT`, `LPRINT USING`	Same as PRINT (there's no printer).
`WRITE items`	Items separated by commas, strings in quotes.
`INPUT ["prompt";] var[, var...]`	With `,` instead of `;` after the prompt, no `?` is shown. Asks again if the input doesn't fit.
`LINE INPUT ["prompt";] var$`	Reads a whole line, commas included.
`CLS`	Clears the screen.
`LOCATE [row][, col][, cursor]`	Rows 1-30, columns 1-80; cursor 0 hides it, 1 shows it. `LOCATE 10, 30: PRINT "HERE"`
`COLOR [fg][, bg]`	PC colours 0-15 (fg 16-31 counts as 0-15), sent as ANSI colours; 7 (fg) and 0 (bg) give the terminal's own. `COLOR 14, 1`
`VIEW PRINT [top TO bottom]`	Text scrolls only within rows top-bottom; `VIEW PRINT` alone undoes it.
`KEY n, x$`, `KEY LIST`, `KEY ON` / `OFF`	Sets or lists the soft keys F1-F10. `KEY ON` shows their labels on row 25, as on a PC. `KEY 5, "FILES" + CHR$(13)`
`WIDTH n`	Accepted; the screen stays 80 columns.

Characters: BASIC uses the IBM PC character set (code page 437), and sends each character to the terminal as its Unicode equivalent in UTF-8: CHR$(219) is a solid block, CHR$(176)-CHR$(178) are shades, CHR$(196) and CHR$(179) are box lines, and control codes show their pictures (CHR$(1) is a smiley, CHR$(3) a heart), except 7 (beep), 8, 9, 10, 12 (clear screen), 13 and 27. 11 moves the cursor home and 28-31 move it right, left, up and down. Text you type is stored as the bytes the terminal sends (UTF-8 for accented letters).

PRINT USING

`PRINT USING fmt$; a, b, ...` prints the values in the fields of `fmt$` (used again from the start if there are more values; other characters are printed as they are):

Field	Meaning
`#`	A digit position. `##.##` prints `3.5` as `3.50`.
`.`	The decimal point.
`,` before the point	Commas between thousands (it also counts as a digit).
`+` first or last	Always show the sign there.
`-` last	A minus after negative numbers.
`**`	Fill the space before the number with `*`.
`$$`	A `$` just before the number. `**$` does both.
`^^^^` (or `^^^^^`)	Exponent form: `##.##^^^^` prints `2.35E+02`.

Field	Meaning
`!`	The first character of a string.
`\ \`	The first n + 2 characters of a string (n spaces between).
`&`	The whole string.
`_`	Prints the next character as it is.

A number too big for its field is printed in full after a `%`. Also `PRINT #n, USING` and after other items: `PRINT "X ="; USING "##.#"; X`.

Functions

Kind	Functions
Maths	`ABS(x)` `INT(x)` (rounds down) `FIX(x)` (cuts off the fraction) `SGN(x)` `SQR(x)` `SIN(x)` `COS(x)` `TAN(x)` `ATN(x)` `EXP(x)` `LOG(x)` `CINT(x)` `CSNG(x)` `CDBL(x)`
Random	`RND` or `RND(1)`: the next number (0 to 1); `RND(0)` repeats the last one; `RND(-n)` reseeds. RUN always starts the same sequence: use `RANDOMIZE` for a different one.
Strings	`LEN(s$)` `LEFT$(s$,n)` `RIGHT$(s$,n)` `MID$(s$,start[,n])` `CHR$(n)` `ASC(s$)` `STR$(x)` `VAL(s$)` `INSTR([start,] s$, find$)` `STRING$(n, c or c$)` `SPACE$(n)` `HEX$(n)` `OCT$(n)` `UCASE$(s$)` `LCASE$(s$)`
Keyboard	`INKEY$`: the key waiting, or "". Cursor and function keys give CHR$(0) and the PC scan code: Up CHR$(0)+CHR$(72), Down 80, Left 75, Right 77, F1-F10 59-68, Home 71, End 79, PgUp 73, PgDn 81, Ins 82, Del 83. `INPUT$(n)`: reads n keys without showing them.
Screen	`POS(0)` (cursor column), `CSRLIN` (cursor row)
System	`TIMER` (seconds since start-up), `FRE(0)` (free memory), `DATE$`, `TIME$`, `ENVIRON$("name")` or `ENVIRON$(n)`, `PEEK(offset)`
Errors	`ERR`, `ERL` (see Error trapping), `ERDEV`, `ERDEV$`, `EXTERR(n)` (always 0 / "")
Sound	`PLAY(0)`: notes still waiting in the background queue
Files	`EOF(n)` `LOF(n)` `LOC(n)` `INPUT$(count, [#]n)` `FREEFILE` `DIR$(pattern$[, n])` `MKI$` `MKS$` `MKD$` `CVI` `CVS` `CVD` (section 4)
Hardware	`PIN(pin)`, `ADC(pin)`, `TEMP()`, `I2CREAD(...)`, `I2CREAD$(...)`, `SPI$(...)` (section 6)

Examples:

```
PRINT INT(-2.5), FIX(-2.5), CINT(2.5)        ' -3  -2  3
PRINT INSTR("HELLO", "L"), MID$("HELLO", 2, 3) '  3  ELL
PRINT HEX$(255); " "; STRING$(5, "*")        ' FF *****
K$ = INKEY$: IF K$ = CHR$(0) + CHR$(72) THEN PRINT "UP"
```

Error trapping

Statement	Notes
`ON ERROR GOTO line`	Sends errors in a running program to that line; `ON ERROR GOTO 0` turns it off (inside the handler, it reports the error and stops).
`RESUME`, `RESUME NEXT`, `RESUME line`	Ends the handler: retries the statement that failed, goes on after it, or goes to the line.
`ERROR n`	Raises error `n` (1-255): a test, or a program's own error codes.

```
10 ON ERROR GOTO 100
20 OPEN "DATA.TXT" FOR INPUT AS #1
30 PRINT "opened": END
100 IF ERR = 53 THEN PRINT "no file": RESUME 30
110 ON ERROR GOTO 0
```

A program that reaches its end while handling an error stops with `No RESUME`. Ctrl-C can't be trapped.

Events

`ON KEY`, `ON TIMER`, `ON PLAY` and `ON COM` jump to a subroutine when something happens; they are checked between statements. `ON`, `OFF` and `STOP` switch an event: one that happens while it is STOPped (or while its own subroutine runs) is remembered and handled once it is back ON. RETURN from the subroutine continues the program where it was.

Statement	When
`ON KEY(n) GOSUB line`, `KEY(n) ON` / `OFF` / `STOP`	Key `n` is pressed: 1-10 are F1-F10, 11 Up, 12 Left, 13 Right, 14 Down.
`ON TIMER(s) GOSUB line`, `TIMER ON` / `OFF` / `STOP`	Every `s` seconds (1-86400).
`ON PLAY(n) GOSUB line`, `PLAY ON` / `OFF` / `STOP`	The background music queue drops below `n` notes.
`ON COM(1) GOSUB line`, `COM(1) ON` / `OFF` / `STOP`	Something arrives on the open COM1.

```
10 ON TIMER(1) GOSUB 100: TIMER ON
20 ON KEY(1) GOSUB 200: KEY(1) ON
30 GOTO 30
100 PRINT TIME$: RETURN
200 PRINT "F1 pressed": RETURN
```

Memory: PEEK and POKE

There's no PC memory. `DEF SEG = segment` with `PEEK(offset)`, `POKE offset, byte`, `BSAVE "file", offset, length` and `BLOAD "file"[, offset]` reach two emulated places; anything else is an Illegal function call:

- `DEF SEG = &HB800` (or `&HB000`): the text screen, two bytes a character. A terminal can't be read back, so PEEK gives spaces (32) and colour 7, and POKE does nothing.
- `DEF SEG = 0` (or `&H40`): BIOS bytes: video mode (&H449, 3), columns (&H44A, 80), rows - 1 (&H484, 29) and the timer tick count (&H46C-&H46E, 18.2 a second, from midnight). The rest read 0, and POKEs there do nothing.

`SCREEN(row, col[, 1])` likewise gives 32 (and with 1, colour 7).

4. The flash disk and data files

The disk

The flash disk is a FAT file system in the board's flash, after the program: about 3 MB on a Pico 2 and 15 MB on a Pico Plus 2. It is formatted (labelled PICOBASIC) the first time the board starts, and keeps its files when the board is switched off or BASIC is loaded again.

- `FILES` lists it and ends with the free space; `FRE(0)` is memory, not disk.
- Names are FAT long names (case is kept, but matching ignores it). Folders work: `MKDIR "GAMES"`, `CHDIR "GAMES"`, `CHDIR ".."`, `RMDIR "GAMES"`, and paths such as `LOAD "GAMES/GUESS"`.
- **Writes are kept in a 4 KB cache** and go to the flash when a file is closed (or the cache moves on to another part of the disk). SAVE closes its file at once; a program's own files are closed by CLOSE, END, RUN, NEW, an error that stops it, and so on. **CLOSE files before switching off**, or the last few KB written can be lost.
- The flash wears out after about 100,000 writes to the same place: plenty for saving programs and logging now and then, but don't rewrite a file many times a second for hours.
- While the flash is being written (a few tens of milliseconds for each 4 KB) the board does nothing else; background PLAY notes may stretch.

The SD card (drive SD:)

A microSD card on SPI is a second drive, `SD:` (or `1:`); the flash disk is `FLASH:` (or `0:`) and stays the current drive at start-up. Cards formatted FAT16, FAT32 (SDHC, up to 32 GB) or exFAT (SDXC) work, as a PC formats them; BASIC doesn't format cards.

Wiring, for the usual microSD breakout (one with its own 3.3 V regulator and level shifter is fine too, powered from 3V3 or VSYS as it says):

Card / module	Pico pin
CLK (SCK)	GP10 (pin 14)
SDI / DI (MOSI)	GP11 (pin 15)
SDO / DO (MISO)	GP12 (pin 16)
DAT1, DAT2 (if the slot has them)	GP13, GP14 (held up: unused in SPI mode)
CS (DAT3)	GP15 (pin 20)
3V3 (VCC)	3V3(OUT) (pin 36)
GND	GND (pin 18)

These are SPI1's pins, and the Cytron Maker Pi Pico's microSD slot is wired just so: it works as it is. Others can be chosen when BASIC is built (see the README). The banner shows the card: `SD card (SD:): 7580 MB, 7571 MB free`, or `SD card (SD:): none`.

- Use it with a drive in front of a name: `SAVE "SD:GAME"`, `LOAD "SD:/GAMES/GUESS"`, `OPEN "SD:LOG.TXT" FOR APPEND AS #1`, `FILES "SD:"`, `FILES "SD:/GAMES/*.BAS"`, `XRECV "SD:PIC.BMP"`.
- `CHDIR "SD:"` makes the card the current drive (then names without a drive are on the card), `CHDIR "SD:/GAMES"` its folder too, and `CHDIR "FLASH:"` goes back.
- Copying between drives: `LOAD "SD:GAME"` then `SAVE "FLASH:GAME"`; for any file, a program that reads one with GET and writes the other with PUT.
- A card can be put in or taken out while BASIC runs: the next use of `SD:` finds the new one. Close files on it first (CLOSE) and don't pull it out while it is being written.
- While a card is in, its pins (GP10-15) and SPI1 are BASIC's: `PIN`, `PWM` and `SPI OPEN` on SPI1 refuse them (`SPI OPEN` on SPI0, its default, is fine). Without a card they are free.

Settings: CONFIG.TXT

`FLASH:/CONFIG.TXT`, if there is one, sets the board's pins at power-up (and after a reset; `SYSTEM` doesn't read it again). Without it the build's defaults hold: the SD card on, on the Maker Pi Pico's pins. Make or change it with `EDIT "FLASH:CONFIG.TXT"`, then reset the board; `CONFIG` shows the settings in force and any problems the file has (the banner shows those too).

```
# Pico BASIC settings (# ' or ; start a comment)
SD = ON            # OFF: no SD card at all (and no start-up delay)
SD_SCK = 10        # the card's pins: one SPI's SCK, MOSI and MISO,
SD_MOSI = 11       #   CS any other pin
SD_MISO = 12
SD_CS = 15
SD_MODE = 3        # SPI mode 0-3 (3 suits most slots; 0 is the usual default)
SD_PULLUPS = 13, 14   # pins held up while the card is used (DAT1, DAT2); none
SPEAKER = 18       # BEEP, SOUND and PLAY's pin; none
NEOPIXEL = 28      # NEOPIXEL OPEN's default pin
```

- Keys and values are any case, with or without spaces; pins can be `18` or `GP18`. Lines it doesn't know are reported and left out.
- The SD card's SPI (SPI0 or SPI1) follows from its pins: SCK on GP2, 6, 10, 14, 18, 22 (26...), MOSI and MISO on the same SPI's pins (MOSI 3, 7, 11, 15..., MISO 0, 4, 8, 12...). Pins that don't fit turn the card off, with the reason in the banner.
- On a board without a card slot, `SD = OFF` saves the moment start-up spends looking for one (BASIC gives up after a tenth of a second when nothing answers at all).
- `examples/CONFIG.TXT` is a copy to start from.

Files from programs

Up to 4 files can be open at once, numbered #1 to #4. File names are relative to the current folder; no extension is added. Each open file uses about 0.8 KB of memory (plus its record buffer for random files).

Statement / function	What it does
`OPEN "file" FOR INPUT | OUTPUT | APPEND | RANDOM AS [#]n [LEN=size]`	Opens a file. OUTPUT creates or empties it, APPEND adds to the end, RANDOM (also the default when FOR is left out) reads and writes fixed-size records (LEN 1-1024, default 128).

Statement / function	What it does
`OPEN "I" | "O" | "A" | "R", [#]n, "file"[, size]`	The older short form.
`CLOSE [[#]n[, [#]n...]]`, `RESET`	Close the given files, or all of them.
`PRINT #n, items`, `PRINT #n, USING fmt$; items`	Like PRINT, into the file. Lines end in CR LF, with no 80-column wrap.
`WRITE #n, items`	Items separated by commas, strings in quotes, so INPUT # reads them back exactly.
`INPUT #n, var[, var...]`	Reads items: a quoted string, or text up to a comma or the end of the line (numbers also stop at a space).
`LINE INPUT #n, var$`	Reads a whole line.
`INPUT$(count, [#]n)`	Reads count characters.
`EOF(n)`	-1 when there is nothing more to read (for a random file: the last GET was past the end).
`LOF(n)`	The file's length in bytes.
`LOC(n)`	For a random file, the last record read or written; otherwise the position in 128-byte blocks.
`FREEFILE`	The lowest free file number.
`FIELD #n, width AS var$[, width AS var$...]`	Divides a random file's record buffer into string fields (at most 16, and no more than the record length).
`LSET var$ = s$` / `RSET var$ = s$`	Puts a string into a field, padded with spaces, aligned left or right. Use these, not LET, for field variables.
`PUT [#]n[, record]`	Writes the record buffer as record `record` (from 1; default: the one after the last).
`GET [#]n[, record]`	Reads a record into the buffer and its FIELD variables.
`MKI$(n)` / `CVI(s$)`	A whole number -32768 to 32767 as a 2-byte string (low byte first), and back.
`MKS$(x)` / `CVS(s$)`	A number as a 4-byte single-precision string, and back.
`MKD$(x)` / `CVD(s$)`	A number as an 8-byte double-precision string, and back.
`DIR$(pattern$[, n])`	The n-th name (1 first) matching the pattern, e.g. `DIR$("*.BAS", 2)`; "" after the last. For going through a folder's files.

Files stay open across lines and statements until CLOSE. RUN, NEW, CLEAR, END, the program finishing, an error that stops the program, and SYSTEM close every file. Ctrl-C does not, so CONT can carry on.

```
10 OPEN "SCORES.TXT" FOR OUTPUT AS #1
20 WRITE #1, "Ann", 120
30 CLOSE #1
40 OPEN "SCORES.TXT" FOR INPUT AS #1
50 WHILE NOT EOF(1): INPUT #1, N$, S: PRINT N$, S: WEND
60 CLOSE #1
```

Random access (a 22-byte record: a 20-character name and a 2-byte number):

```
10 OPEN "PEOPLE.DAT" FOR RANDOM AS #1 LEN = 22
20 FIELD #1, 20 AS NM$, 2 AS AGE$
30 LSET NM$ = "Grace": LSET AGE$ = MKI$(85): PUT #1, 1
40 GET #1, 1: PRINT NM$; CVI(AGE$)
50 CLOSE #1
```

MKS$ and MKD$ use GW-BASIC's own number format (Microsoft Binary Format), so random files written by GW-BASIC on a PC read correctly here, and the other way round. CVI, CVS and CVD need at least 2, 4 and 8 bytes (otherwise Illegal function call).

Differences from GW-BASIC:

- A file can be open for INPUT under several numbers, but not written while it is open elsewhere.
- Ctrl-Z (byte 26) ends a file for INPUT # and INPUT$, as in GW-BASIC, so binary files are best read with random access (GET).

5. Moving files to and from the PC

Pasting and listing

The simplest way to bring a program over: type `NEW`, paste its listing into the terminal at the `Ok` prompt (the numbered lines go into the program), then `SAVE "NAME"`. To take one back: `LOAD` it, `LIST` it, and copy the text from the terminal window. Many terminals can also log what is shown to a file.

If pasted lines get lost, slow the terminal's paste down (a few milliseconds per line): BASIC reads keys between lines, and the USB port holds only a few hundred characters meanwhile.

XMODEM: XRECV and XSEND

These copy any file, text or binary, between the flash disk and the PC over the terminal line:

Statement	What it does
`XRECV "file"`	Receives a file from the terminal's XMODEM send and saves it as `file`, replacing a file of that name. It asks for CRC-16, falling back to checksums for an old sender, and takes 128- or 1K-byte blocks. XMODEM pads the last block with ^Z (byte 26) characters; they are taken off the end. If the transfer fails, the partial file is deleted (a file that was already there is kept if nothing arrived).
`XSEND "file"`	Sends the file to the terminal's XMODEM receive, in 1K blocks with CRC-16, or 128-byte blocks with a checksum if the receiver asks for those.

Type the command; BASIC says `Start the XMODEM send on the terminal now.` (or receive), and you then have a minute to start the transfer on the PC. XMODEM doesn't send the file's name, so give it on both ends. Pressing Ctrl-X a few times in the terminal cancels a transfer; a failed one gives `Device I/O error` after a line saying why. When it ends, BASIC prints e.g. `Received 1 file, 329 bytes.`

```
XRECV "GAME.BAS"
(send GAME.BAS from the terminal)
LOAD "GAME"
RUN
```

Starting the PC's side:

- **minicom:** Ctrl-A S, choose `xmodem`, then the file, to send it to XRECV. Ctrl-A R, `xmodem` and a name receives from XSEND. (minicom uses lrzsz's `sx` and `rx`: `sudo apt install lrzsz`.)
- **picocom:** start it with `picocom --send-cmd "sx -vv" --receive-cmd "rx -vv" /dev/ttyACM1`, then Ctrl-A Ctrl-S sends a file and Ctrl-A Ctrl-R receives one (type the name to save it as).
- **Tera Term:** File > Transfer > XMODEM > Send... or Receive... (pick CRC or 1K).
- **From a shell**, with the terminal program closed (or disconnected) so it doesn't take the bytes:

```
stty -F /dev/ttyACM1 raw -echo
sx GAME.BAS < /dev/ttyACM1 > /dev/ttyACM1      # to XRECV
rx GAME.BAS < /dev/ttyACM1 > /dev/ttyACM1      # from XSEND
```

(with the board's port in place of `/dev/ttyACM1`).

The PC's receiver may keep XMODEM's ^Z padding (up to 1 KB of byte 26 at the end). lrzsz's `rx` does, unless run as `rx -a` for a text file (which also converts line ends). Because BASIC removes it, a binary file that really ends in ^Z bytes loses them on the way in.

6. Sound

A square wave on **GP18** (another pin if BASIC was built with a different `SOUND_PIN`), for a piezo buzzer or a small speaker through a resistor.

Statement	What it does
`BEEP`	A short beep (800 Hz for a quarter of a second).
`SOUND freq, ticks`	A tone of freq Hz (37-32767; 32767 is silence) for `ticks` (18.2 make a second). After `PLAY "MB"` it goes to the background queue; ticks 0 stops the sound playing. `SOUND 440, 18`
`PLAY "string"`	Music in GW-BASIC's music language (below). `PLAY "T160 O3 L8 CDEFGAB > C"`
`PLAY(0)`	Notes still waiting in the background queue.

PLAY's commands:

Command	Meaning
`A-G` [`#`, `+` or `-`] [length] [`.`]	A note (sharp or flat), optionally with its own length; each dot adds half again.
`N n`	Note number n (1-84; 0 is a rest).
`O n`, `>`, `<`	Octave 0-6 (default 4; octave 3 starts at middle C), up one, down one.
`L n`	Length for following notes: 1 whole, 4 quarter (default), up to 64.
`P n`	A pause, length as for L.
`T n`	Tempo, 32-255 quarter notes a minute (default 120).
`MN` `ML` `MS`	Normal (7/8 of each note sounds), legato (all), staccato (3/4).
`MF` `MB`	Foreground (PLAY waits until the music ends; the default) or background (up to 32 notes queue up and the program goes on).
`=variable;`	A number from a variable: `PLAY "O=OC;C"`.

The settings last from one PLAY to the next; RUN resets them. `X` (substrings through VARPTR$) isn't supported. The tones are plain square waves of one voice, so a note waits for the one before it.

7. Hardware: GPIO, PWM, ADC, I2C, SPI, COM1

Pins

Pins are given by their GP number; `LED` means the board's LED (GP25). The Pico and Pico 2 have GP0-29 (GP23-25 and GP29 aren't on the header: power control, VBUS sense, the LED and VSYS / 3); the Pico Plus 2 has GP0-47.

Pins BASIC itself uses are refused, e.g. `GP18 is used by the speaker (BEEP, SOUND, PLAY)` followed by `Illegal function call`:

Pins	Used by
GP18	the speaker
GP10-15	the SD card, while one is in
GP47 (Pico Plus 2)	the PSRAM
COM1's, I2C's and SPI's pins	while they are open

Statement / function	What it does
`PINMODE pin, INPUT | OUTPUT | PULLUP | PULLDOWN`	Sets a pin up (or modes 0-3). `PINMODE 16, PULLUP`
`PIN pin, level`	Drives a pin 0 or 1, making it an output first if needed. `PIN LED, 1`
`PIN(pin)`	Reads a pin, making an unused pin an input. `IF PIN(16) = 0 THEN PRINT "PRESSED"`
`PWM pin, freq[, duty]`	PWM at 8 Hz-10 MHz with a duty of 0-100 % (default 50). `PWM pin, 0` stops it. `PWM 16, 1000, 25`
`SERVO pin, angle`	A hobby servo, 0-180 degrees (0.5-2.5 ms pulses at 50 Hz). `SERVO 16, 90`
`ADC(pin)`	An analog input, 0-4095 for 0-3.3 V: GP26-29 on the Pico and Pico 2, GP40-46 on the Pico Plus 2. `V = ADC(26) * 3.3 / 4095`
`TEMP()`	The chip's temperature in °C (uncalibrated: within a few degrees).
`PINS`	Lists every pin and what it is doing.
`NEOPIXEL ...`	WS2812B LEDs (below).

- PWM pins come in pairs that share one frequency (GP0/1, GP2/3 and so on); changing one's frequency prints a warning naming its partner. The chip has only 8 slices for GP0-31, so GP16-31 use the same ones as GP0-15: GP2 and GP18 (and GP3 and GP19) give the very same signal, with no warning. The speaker's slice (GP18) is shared with GP2, GP3 and GP19: PWM there and sound interfere.
- `RUN`, `NEW` and `SYSTEM` set every pin BASIC used back to an input and stop PWM. `END`, errors and Ctrl-C leave pins as they are, so they can be checked in direct mode.
- `PIN`, `ADC` and `TEMP` followed by `(` are functions, so arrays can't use those names.

I2C and SPI

The chip's own I2C and SPI controllers, for sensors, displays, memory chips and the like. The pins choose the controller: I2C wants SDA on an even pin and SCL on the next one (I2C0 or I2C1 by the pin numbers); SPI wants SCK, MOSI and MISO from one group of four (MISO, CS, SCK, MOSI: GP8-11 is SPI1, GP16-19 SPI0 ...). RUN, NEW and SYSTEM close them.

Statement / function	What it does
`I2C OPEN [sda, scl[, khz]]`	Starts I2C, by default on **GP2 (SDA) and GP3 (SCL)**, I2C1, at 100 kHz (10-1000). Turns on the pins' weak pull-ups; devices should have their own.
`I2C WRITE addr, item[, item...][, NOSTOP]`	Sends bytes to the device at `addr` (0-127): numbers 0-255 and strings (their bytes). With `NOSTOP` the bus is kept for a read that follows.
`I2CREAD$(addr, n[, reg])`	Reads `n` bytes (1-255); with `reg`, first writes that register number (without a stop).
`I2CREAD(addr[, reg])`	Reads one byte.
`I2C SCAN`	Prints the addresses that answer (8-119).
`I2C CLOSE`	Stops it and frees the pins.
`SPI OPEN [sck, mosi, miso[, khz[, mode]]]`	Starts SPI, by default on **GP6 (SCK), GP7 (MOSI), GP4 (MISO)**, SPI0, at 1000 kHz (1-50000), mode 0 (0-3); 8-bit words, most significant bit first. Chip select is any pin you drive with `PIN` (**GP5** is free for it). (SPI1 is the SD card's.)
`SPI WRITE item[, item...]`	Sends bytes (numbers 0-255 and strings), ignoring what comes back.
`SPI$(out$)`	Sends `out$` and returns the bytes that came back meanwhile: `SPI$(STRING$(4, 0))` reads 4.
`SPI CLOSE`	Stops it and frees the pins.

A device that doesn't answer gives `Device I/O error` (57); using I2C or SPI before OPEN gives `Device unavailable` (68). Both can be caught with ON ERROR.

```
10 I2C OPEN: I2C SCAN
20 T = I2CREAD(&H48, 0): PRINT "LM75 at &H48:"; T
30 PIN 5, 1: SPI OPEN: PIN 5, 0: SPI WRITE &H9F
40 ID$ = SPI$(STRING$(3, 0)): PIN 5, 1: PRINT LEN(ID$); "bytes"
```

NeoPixels: WS2812B LEDs

Strips, rings and single WS2812B ("NeoPixel") LEDs, on any free pin, driven by one of the chip's PIO state machines. Colours are kept in a buffer and sent with `NEOPIXEL SHOW`.

Statement	What it does
`NEOPIXEL OPEN [pin[, count[, order$]]]`	Sets up `count` LEDs (1-1024, default 1) on `pin` (default **GP28**, the Maker Pi Pico's RGB LED), all off. `order$` is `"GRB"` (WS2812B, the default) or `"RGB"` for strips that take red first. Opening again replaces the strip.
`NEOPIXEL n, r, g, b`	LED `n` (0 first) to red, green and blue 0-255, in the buffer.
`NEOPIXEL FILL r, g, b`	Every LED to one colour.

Statement	What it does
`NEOPIXEL SHOW`	Sends the buffer to the LEDs.
`NEOPIXEL BRIGHT level`	Brightness 0-255 (255 at first) for the next SHOWs: full white is bright and draws 60 mA a LED.
`NEOPIXEL CLEAR`	All off, shown at once.
`NEOPIXEL CLOSE`	Frees the pin and the state machine.

- The pin is refused to `PIN`, `PWM` and the rest while the strip is open; `PINS` shows it. RUN, NEW and SYSTEM close it (the LEDs keep their last colours: `NEOPIXEL CLEAR` first to turn them off).
- Using it before `NEOPIXEL OPEN` gives `Device unavailable` (68).
- WS2812B LEDs want 5 V but take the Pico's 3.3 V data on a short wire. Power a strip of more than a few LEDs from its own 5 V supply, join its GND to the Pico's, and keep `BRIGHT` modest.

  10 NEOPIXEL OPEN 16, 8: NEOPIXEL BRIGHT 40 20 FOR I = 0 TO 7: NEOPIXEL I, I * 32, 0, 255 - I * 32: NEXT 30 NEOPIXEL SHOW

`examples/NEOPIXEL.BAS` shows red, green and blue, then a rainbow; `examples/NEODEMO.BAS` has eight effects to step through.

Serial port (COM1)

A serial port on the chip's UART1 (the console is on USB). It works like a file: open it with a "COM1:" name, then use PRINT #, WRITE #, INPUT #, LINE INPUT # and INPUT$ on it.

```
OPEN "COM1:[speed][,parity][,data][,stop][,options]" AS #n
```

- **Settings:** speed 75-921600 baud; parity N, E or O; data bits 5-8; stop bits 1 or 2. As in GW-BASIC, anything left out defaults to 300 baud, even parity, 7 data bits and 1 stop bit, so usually you give them all: `OPEN "COM1:9600,N,8,1" AS #1`.
- **Pins:** GP8 (TX) and GP9 (RX) unless `TXn` / `RXn` options choose others: TX on GP4, 8 or 20 and RX on GP5, 9 or 21 (`OPEN "COM1:9600,N,8,1,TX8,RX9" AS #1`). While it is open, those pins are refused to PIN and PWM.
- **Options:** `LF` sends an LF after each CR. `RS`, `CS`, `DS`, `CD`, `PE`, `BIN` and `ASC` are accepted and do nothing (there are no handshake lines).
- **Sending:** PRINT # ends lines with CR (CR LF with LF). It waits while the UART is busy.
- **Receiving:** bytes go into a 4 KB buffer in the background, so none are lost while the program is busy. `LOC(n)` is how many are waiting, `EOF(n)` is -1 when none are, and `LOF(n)` is the room left. INPUT #, LINE INPUT # and INPUT$ wait for data (Ctrl-C stops the wait). A line ends at CR (an LF after it is skipped). `ON COM(1) GOSUB` runs when bytes arrive.
- **Levels:** 3.3 V logic. Use a level shifter or an RS-232 adapter for RS-232 or 5 V devices, and connect GND too.
- Only one COM port can be open, and GET/PUT/FIELD don't apply to it.

`examples/SERIAL.BAS` is a small terminal: wire GP8 to GP9 and what you type comes straight back.

The Cytron Maker Pi Pico

BASIC's defaults fit this carrier board:

- **microSD slot:** drive `SD:` as it is (SPI1, GP10-15).
- **Buzzer:** GP18, for BEEP, SOUND and PLAY (its mute switch turns it off); the 3.5 mm audio jack is on GP18 (left) and GP19 (right).
- **Buttons:** GP20, GP21 and GP22, pressed = 0. Turn their pull-ups on first (`PINMODE 20, PULLUP` and so on), or a released button can read 0 now and then; then `PIN(20)` is 0 while the button is pressed.
- **RGB LED:** a WS2812B on GP28: `NEOPIXEL OPEN` (with no arguments) then `NEOPIXEL 0, 255, 0, 0: NEOPIXEL SHOW` lights it red.
- **LEDs:** every GPIO has one, so they show what the pins are doing.
- **Free for your own use:** COM1 (GP8/GP9), `SPI OPEN` (GP4-7), I2C (GP2/GP3) and the Grove connectors on those pins.

8. Memory

- **Pico Plus 2:** the PSRAM is found at start-up (the banner shows it). Blocks of 512 bytes or more (arrays, mostly) go in the 8 MB PSRAM, and small ones (strings, program lines, variables) in the faster SRAM until it runs low, so programs get about 8.4 MB.
- **Pico 2:** everything is in SRAM, about 450 KB.
- **Pico (RP2040):** everything is in SRAM, about 190 KB: room for a long program, but `DIM A(20000)` (8 bytes an element) is about the most one array can have.
- `FRE(0)` (and the banner) give the memory free. A program line is refused if it would leave less than 16 KB for data, and any allocation if it would leave less than 4 KB.
- BASIC runs on a 32 KB stack: very deeply nested expressions give `Out of memory` rather than crashing.

9. Errors

Errors print as `<message> in <line>` (`in line <n>` for a program without line numbers; just the message in direct mode), and `ERR` gives the code:

Code	Message	Code	Message
1	NEXT without FOR	26	FOR without NEXT
2	Syntax error	27	Out of paper
3	RETURN without GOSUB	29	WHILE without WEND
4	Out of DATA	30	WEND without WHILE
5	Illegal function call	50	FIELD overflow
6	Overflow	51	Internal error
7	Out of memory	52	Bad file number
8	Undefined line number	53	File not found
9	Subscript out of range	54	Bad file mode
10	Duplicate Definition	55	File already open
11	Division by zero	57	Device I/O error
12	Illegal direct	58	File already exists
13	Type mismatch	61	Disk full
14	Out of string space	62	Input past end
15	String too long	63	Bad record number
16	String formula too complex	64	Bad file name
17	Can't continue	66	Direct statement in file
18	Undefined user function	67	Too many files
19	No RESUME	68	Device unavailable
20	RESUME without error	69	Communication buffer overflow
22	Missing operand	70	Permission denied
23	Line buffer overflow	71	Disk not ready
24	Device Timeout	72	Disk Media Error
25	Device Fault	73	Feature not available
		74	Rename across disks
		75	Path/File Access Error
		76	Path not found
		80	Undefined label
		81	Duplicate label

Codes BASIC itself never raises can still be raised with `ERROR n`; unknown codes print `Unprintable error`. Ctrl-C prints `Break in <line>`.

10. Limits

- **Lines and strings:** program lines and strings are up to 255 characters.
- **Nesting:** FOR, GOSUB and WHILE can each be nested up to 32 deep.
- **Files:** 4 open at once; random records up to 1024 bytes; 16 FIELD variables per file.
- **CHAIN:** up to 64 COMMON variables.
- **History:** the last 8 lines typed.
- **The screen:** 80 x 30 as far as BASIC knows (LOCATE, CSRLIN, the KEY line); a bigger terminal window is fine.

11. Example programs

The source's `examples` folder has these; send them over with `XRECV` (section 5) or paste them in.

Program	What it does
PRIMES.BAS	prints the primes below 1000
LABELS.BAS	a menu program without line numbers: GOSUB, ON GOSUB, RESTORE, ON ERROR GOTO and RESUME with labels
GUESS.BAS	a guess-the-number game
BOXES.BAS	coloured boxes drawn with the PC line-drawing characters
FILES.BAS	writes, appends and reads back a text file, and stores and reads records (MKI$/CVI) in a random-access file
BLINK.BAS	flashes the LED
FADE.BAS	fades the LED with PWM
ANALOG.BAS	shows the ADC inputs and the chip temperature (GP26-29; change line 16 to `FIRST = 40: LAST = 46` on a Pico Plus 2)
SERIAL.BAS	a small terminal on COM1 (GP8 TX, GP9 RX)
I2CSCAN.BAS	lists the I2C devices on GP2/GP3 and reads the first one's registers 0-7
NEODEMO.BAS	eight NeoPixel effects (rainbow, breathe, colour wipe, chase, candle, sparkle, police, heartbeat), changing every 8 seconds; Space/N, P, + and - (or the Maker Pi Pico's buttons GP20-22) to choose, Q quits; one line sets a strip's pin and length
NEOPIXEL.BAS	red, green, blue, then a rainbow on WS2812B LEDs (the Maker Pi Pico's RGB LED on GP28, or a strip)
SPIFLASH.BAS	reads an SPI flash chip's JEDEC ID and first 16 bytes (SPI0 on GP4-7)

For example, from a shell (terminal closed) and then the terminal:

```
XRECV "PRIMES.BAS"                          (in BASIC)
sx examples/PRIMES.BAS < /dev/ttyACM1 > /dev/ttyACM1
RUN "PRIMES"                                (in BASIC)
```

12. Command reference

Every statement, command, function and operator, A to Z. Each entry gives its form (`[...]` is optional, `|` separates choices), what it does, its limits and errors, an example and what the terminal then shows. Numbers print with a space before them (or a `-`) and one after.

Index by topic

Programs and the disks: AUTO, BYE, CHAIN, CHDIR, CLEAR, COMMON, CONFIG, CONT, DELETE, EDIT, FILES, FORMAT, KILL, LIST, LLIST, LOAD, MERGE, MKDIR, NAME, NEW, NUMBERS, RENUM, RESET, RMDIR, RUN, SAVE, SYSTEM, TROFF, TRON, XRECV, XSEND

Flow of control and errors: END, ERL, ERR, ERROR, EXIT FOR, FOR, GOSUB, GOTO, IF, NEXT, ON ERROR GOTO, ON...GOSUB, ON...GOTO, PAUSE, REM, RESUME, RETURN, STOP, WEND, WHILE

Variables and data: DATA, DEF FN, DEF SEG, DEFDBL, DEFINT, DEFSNG, DEFSTR, DIM, ERASE, LET, MID$ =, OPTION BASE, RANDOMIZE, READ, RESTORE, SWAP

The screen and keyboard: ?, CLS, COLOR, CSRLIN, INKEY$, INPUT, INPUT$, KEY, LINE INPUT, LOCATE, LPRINT, POS, PRINT, PRINT USING, SPC, TAB, VIEW PRINT, WIDTH, WRITE

Numbers: ABS, ATN, CDBL, CINT, COS, CSNG, EXP, FIX, INT, LOG, RND, SGN, SIN, SQR, TAN

Strings: ASC, CHR$, HEX$, INSTR, LCASE$, LEFT$, LEN, MID$, OCT$, RIGHT$, SPACE$, STR$, STRING$, UCASE$, VAL

Data files: CLOSE, CVD, CVI, CVS, DIR$, EOF, FIELD, FREEFILE, GET, INPUT #, LINE INPUT #, LOC, LOF, LSET, MKD$, MKI$, MKS$, OPEN, PRINT #, PUT, RSET, WRITE #

Events: COM(1), KEY(n), ON COM, ON KEY, ON PLAY, ON TIMER, PLAY ON, TIMER ON

Sound: BEEP, PLAY, PLAY(n), SOUND

Pins and hardware: ADC, I2C, I2CREAD, I2CREAD$, NEOPIXEL, PIN, PINMODE, PINS, PWM, SERVO, SPI, SPI$, TEMP

System: BLOAD, BSAVE, DATE$, ENVIRON, ENVIRON$, ERDEV, EXTERR, FRE, PEEK, POKE, SCREEN(r, c), TIME$, TIMER

Operators: + - * / ^, = <> < > <= >=, AND, EQV, IMP, NOT, OR, XOR, \ and MOD

+ - * / ^

Operator · `a + b, a - b, a * b, a / b, a ^ b`

Add, subtract, multiply, divide, power. `+` also joins strings. `-x` negates. Calculations are in double precision.

- Errors: `Division by zero` (11), `Overflow` (6), `Type mismatch` (13) mixing strings and numbers.

Example:

```
PRINT 2 ^ 10; 7 / 2; "AB" + "CD"
```

Shows:

```
 1024  3.5 ABCD
```

See also: \ and MOD

= <> < > <= >=

Operator · `a = b, a <> b, a < b, a > b, a <= b, a >= b`

Comparisons, of numbers or of strings (by character codes): -1 for true, 0 for false.

Example:

```
PRINT 3 > 2; "A" < "B"; 1 = 2
```

Shows:

```
-1 -1  0
```

See also: IF, AND

?

Statement · `? items`

Short for PRINT.

Example:

```
? 1 + 1
```

Shows:

```
 2
```

See also: PRINT

ABS

Function · `ABS(x)`

The absolute value of `x` (`x` without its sign).

Example:

```
PRINT ABS(-3.5)
```

Shows:

```
 3.5
```

See also: SGN

ADC

Function · `ADC(pin)`

Reads an analog input: 0-4095 for 0-3.3 V. GP26-29 on the Pico and Pico 2 (GP29 is VSYS / 3), GP40-46 on the Pico Plus 2.

- Errors: `Illegal function call` (5) for a pin without one.

Example:

```
PRINT ADC(26) * 3.3 / 4095
```

Shows:

```
 1.65040293
```

See also: TEMP, PIN

AND

Operator · `a AND b`

Bitwise AND of two integers: true (-1) only if both are. Also masks bits: `X AND 15`.

Example:

```
PRINT 12 AND 10; (1 < 2) AND (2 < 3)
```

Shows:

```
 8 -1
```

See also: OR, NOT

ASC

Function · `ASC(s$)`

The code (0-255) of the first character of `s$`.

- Errors: `Illegal function call` (5) for "".

Example:

```
PRINT ASC("A")
```

Shows:

```
 65
```

See also: CHR$

ATN

Function · `ATN(x)`

The arctangent of `x`, in radians (-π/2 to π/2).

Example:

```
PRINT ATN(1) * 4
```

Shows:

```
 3.14159265
```

See also: TAN, SIN, COS

AUTO

Command · `AUTO [start][, step]`

Numbers the lines you type: each new line starts with the next number, so you just type the statement.

- `start` and `step` default to 10 and 10.
- A `*` after the number means that line already exists: Enter keeps it as it is.
- An empty line where no line exists, or Ctrl-C, stops it.

Example:

```
AUTO 100, 5
```

Shows:

```
100
```

See also: RENUM, DELETE

BEEP

Statement · `BEEP`

A short beep: 800 Hz for a quarter of a second, on the speaker pin (GP18, the Maker Pi Pico's buzzer).

Example:

```
BEEP
```

See also: SOUND, PLAY

BLOAD

Statement · `BLOAD file$[, offset]`

Loads a file saved by BSAVE (GW-BASIC's 7-byte header) into the emulated memory (DEF SEG).

See also: BSAVE, DEF SEG

BSAVE

Statement · `BSAVE file$, offset, length`

Saves `length` bytes of the emulated memory (DEF SEG) to a file.

See also: BLOAD

BYE

Command · `BYE`

The same as SYSTEM.

See also: SYSTEM

CDBL

Function · `CDBL(x)`

`x` as a double-precision number.

Example:

```
PRINT CDBL(1) / 3
```

Shows:

```
 .333333333
```

See also: CSNG, CINT

CHAIN

Statement · `CHAIN [MERGE] file$[, [line][, ALL][, DELETE range]]`

Runs another program from a program, keeping the files that are open.

- Only the variables named in COMMON are passed on, or all of them with `ALL`.
- `line` is where the new program starts (default its first line).
- `CHAIN MERGE` adds the file's lines to the program (after deleting `range`) and keeps every variable.
- `.BAS` is added if the name has no extension.

Example:

```
10 COMMON SCORE
20 SCORE = 100: CHAIN "PART2"
RUN
```

See also: COMMON, MERGE, RUN

CHDIR

Statement · `CHDIR path$`

Makes a folder the current one, or a drive the current drive.

- `CHDIR "GAMES"`, `CHDIR ".."`, `CHDIR "/"` move between folders.
- `CHDIR "SD:"` makes the SD card the current drive, `CHDIR "SD:/GAMES"` its folder too, `CHDIR "FLASH:"` goes back.
- Errors: `Path not found` (76).

Example:

```
MKDIR "GAMES": CHDIR "GAMES": FILES
```

Shows:

```
File not found
3080192 Bytes free
```

See also: MKDIR, RMDIR, FILES

CHR$

Function · `CHR$(n)`

The character with code `n` (0-255), in the IBM PC character set (see Characters).

- Errors: `Illegal function call` (5) out of range.

Example:

```
PRINT CHR$(65); CHR$(219); CHR$(1)
```

Shows:

```
A☺
```

See also: ASC, STRING$

CINT

Function · `CINT(x)`

`x` rounded to the nearest whole number (halves away from 0), -32768 to 32767.

- Errors: `Overflow` (6) out of range.

Example:

```
PRINT CINT(2.5); CINT(-2.5); CINT(2.4)
```

Shows:

```
 3 -3  2
```

See also: INT, FIX

CLEAR

Statement · `CLEAR [, n[, m]]`

Deletes every variable and array, closes every file, and forgets DATA, FOR, GOSUB and WHILE positions and event traps. The program stays.

- Any numbers after it (GW-BASIC's memory sizes) are accepted and ignored.

Example:

```
A = 5: CLEAR: PRINT A
```

Shows:

```
 0
```

See also: NEW, ERASE

CLOSE

Statement · `CLOSE [[#]n[, [#]n...]]`

Closes the files numbered, or all of them. Writes still in memory go to the disk.

Example:

```
CLOSE #1
```

See also: OPEN, RESET

CLS

Statement · `CLS [n]`

Clears the terminal and puts the cursor at the top left.

- `n` (0-2) is accepted for old programs.

See also: LOCATE, COLOR

COLOR

Statement · `COLOR [fg][, [bg][, border]]`

Sets the text colours, as PC colours 0-15 sent as ANSI codes: 0 black, 1 blue, 2 green, 3 cyan, 4 red, 5 magenta, 6 brown, 7 white, 8-15 bright. 7 (fg) and 0 (bg) give the terminal's own colours.

- fg 16-31 counts as 0-15 (no blinking). `border` is ignored.
- Errors: `Illegal function call` (5) out of range.

Example:

```
COLOR 14, 1: PRINT "YELLOW ON BLUE": COLOR 7, 0
```

Shows:

```
YELLOW ON BLUE
```

See also: CLS

COM(1)

Statement · `COM(1) ON | OFF | STOP`

Turns the COM1 event (ON COM) on, off, or holds it (remembered until ON).

See also: ON COM

COMMON

Statement · `COMMON var[, var...]`

Names the variables CHAIN passes to the next program. Arrays are written with empty brackets: `COMMON A(), N$`.

- Up to 64 names.

Example:

```
COMMON SCORE, NAME$
```

See also: CHAIN

CONFIG

Command · `CONFIG`

Shows the board's settings: whether the SD card is used and its pins, the speaker's pin and NEOPIXEL's default pin, from FLASH:/CONFIG.TXT or the build's defaults, and any problems CONFIG.TXT has.

- Change them with `EDIT "FLASH:CONFIG.TXT"`; they take effect at the next power-up or reset (see the manual's Settings section).

Example:

```
CONFIG
```

Shows:

```
Settings (FLASH:/CONFIG.TXT):
  SD = OFF
  SPEAKER = GP18
  NEOPIXEL = GP28 (NEOPIXEL OPEN's default)
```

See also: EDIT, NEOPIXEL, BEEP

CONT

Command · `CONT`

Carries on with the program after Ctrl-C or STOP, from where it stopped.

- Errors: `Can't continue` (17) after the program has been changed or after an error.

See also: STOP, END

COS

Function · `COS(x)`

The cosine of `x` radians.

Example:

```
PRINT COS(0)
```

Shows:

```
 1
```

See also: SIN, TAN

CSNG

Function · `CSNG(x)`

`x` as a single-precision number (7 digits).

Example:

```
PRINT CSNG(1#/3)
```

Shows:

```
 .3333333
```

See also: CDBL

CSRLIN

Function · `CSRLIN`

The row the cursor is on, 1-30.

Example:

```
LOCATE 10, 1: PRINT CSRLIN
```

Shows:

```
 10
```

See also: POS, LOCATE

CVD

Function · `CVD(s$)`

The double-precision number in an 8-byte string made by MKD$ (GW-BASIC's format).

- Errors: `Illegal function call` (5) for a string shorter than 8.

See also: MKD$

CVI

Function · `CVI(s$)`

The integer in a 2-byte string made by MKI$ (low byte first).

Example:

```
PRINT CVI(MKI$(1234))
```

Shows:

```
 1234
```

See also: MKI$

CVS

Function · `CVS(s$)`

The single-precision number in a 4-byte string made by MKS$.

Example:

```
PRINT CVS(MKS$(2.5))
```

Shows:

```
 2.5
```

See also: MKS$

DATA

Statement · `DATA item[, item...]`

Holds numbers and strings for READ. Strings may be quoted (needed if they hold commas or start with spaces).

Example:

```
10 DATA 3, "RED", 2.5
```

See also: READ, RESTORE

DATE$

Statement / function · `DATE$ = "mm-dd-yyyy" | DATE$`

Sets the date, or gives it as "mm-dd-yyyy". `-` or `/` and two-digit years are accepted. There is no battery clock: it starts at 01-01-1970 at power-up.

Example:

```
DATE$ = "10-31-2026": PRINT DATE$
```

Shows:

```
10-31-2026
```

See also: TIME$

DEF FN

Statement · `DEF FNname[(arg[, arg...])] = expr`

Defines a one-line function, called as FNname(...). String functions end in $.

- Only in a program line: in direct mode it is `Illegal direct` (12).
- Errors: `Undefined user function` (18) for one not defined.

Example:

```
10 DEF FNSQ(X) = X * X: PRINT FNSQ(7)
RUN
```

Shows:

```
 49
```

See also: GOSUB

DEF SEG

Statement · `DEF SEG [= segment]`

Chooses the memory segment PEEK, POKE, BSAVE and BLOAD use: only `&HB800` (the text screen) and `0` (BIOS data) are emulated.

Example:

```
DEF SEG = 0: PRINT PEEK(&H44A)
```

Shows:

```
 80
```

See also: PEEK, POKE

DEFDBL

Statement · `DEFDBL letters`

Names starting with these letters (and no type character) are double precision: `DEFDBL A-C, X`.

- RUN, NEW and CLEAR reset it.

Example:

```
DEFDBL D: D = 1 / 3: PRINT D
```

Shows:

```
 .333333333
```

See also: DEFINT, DEFSNG, DEFSTR

DEFINT

Statement · `DEFINT letters`

Names starting with these letters are integers (-32768 to 32767): `DEFINT A-Z` makes all plain names integers.

Example:

```
DEFINT I: I = 2.7: PRINT I
```

Shows:

```
 3
```

See also: DEFDBL, DEFSNG, DEFSTR

DEFSNG

Statement · `DEFSNG letters`

Names starting with these letters are single precision (the default).

See also: DEFINT, DEFDBL

DEFSTR

Statement · `DEFSTR letters`

Names starting with these letters are strings, without a `$`.

Example:

```
DEFSTR S: S = "HI": PRINT S
```

Shows:

```
HI
```

See also: DEFINT

DELETE

Command · `DELETE range`

Deletes program lines. `range` is `n`, `n-`, `-n` or `n-m`.

- Errors: `Illegal function call` (5) if there is nothing in the range.

Example:

```
DELETE 100-190
```

See also: NEW, AUTO

DIM

Statement · `DIM name(size[, size...])[, ...]`

Makes an array with subscripts 0 (or 1 after OPTION BASE 1) to `size`, up to 8 dimensions. An array used without DIM gets 0-10 in each dimension.

- Errors: `Duplicate Definition` (10) if it exists; `Subscript out of range` (9) for a subscript too big; `Out of memory` (7).

Example:

```
DIM A(5), B$(2, 3): A(5) = 7: PRINT A(5)
```

Shows:

```
 7
```

See also: ERASE, OPTION BASE

DIR$

Function · `DIR$(pattern$[, n])`

The `n`-th (default 1st) file name matching the pattern, e.g. `DIR$("*.BAS", 2)`; "" after the last. For going through a folder's files.

Example:

```
10 I = 1
20 F$ = DIR$("*.BAS", I): IF F$ = "" THEN END
30 PRINT F$: I = I + 1: GOTO 20
RUN
```

Shows:

```
PRIMES.BAS
NEODEMO.BAS
```

See also: FILES

EDIT

Command · `EDIT | EDIT file$ | EDIT n | EDIT .`

`EDIT` opens the program in the full-screen editor (see the manual's Full-screen editor section); `EDIT file$` edits any text file there. `EDIT n` (or `.`, the last line typed or the one that failed) puts program line `n` on the input line to change; Enter stores it.

- In the full-screen editor Ctrl-X/C/V cut, copy and paste lines, Ctrl-Z/R undo and redo, Ctrl-S saves and Esc leaves. A program is only taken if every non-blank line starts with a line number (0-65529, 255 characters at most): otherwise nothing is saved and the cursor goes to the bad line.
- Leaving without saving keeps the program as it was. Blank lines are left out.

Example:

```
EDIT 20
```

Shows:

```
20 PRINT "HELLO"
```

See also: LIST, AUTO

END

Statement · `END`

Ends the program and closes its files.

Example:

```
10 PRINT "DONE": END
RUN
```

Shows:

```
DONE
```

See also: STOP

ENVIRON

Statement · `ENVIRON "name=value"`

Sets a string ENVIRON$ gives back (there is no DOS environment).

Example:

```
ENVIRON "USER=KEN"
```

See also: ENVIRON$

ENVIRON$

Function · `ENVIRON$(name$) | ENVIRON$(n)`

The value set for `name$` with ENVIRON, or the n-th "name=value" string; "" if none.

Example:

```
ENVIRON "USER=KEN": PRINT ENVIRON$("USER")
```

Shows:

```
KEN
```

See also: ENVIRON

EOF

Function · `EOF(n)`

-1 when file `n` has nothing more to read (for a random file: the last GET was past the end; for COM1: nothing waiting), else 0.

Example:

```
WHILE NOT EOF(1): LINE INPUT #1, L$: PRINT L$: WEND
```

See also: LOF, LOC

EQV

Operator · `a EQV b`

Bitwise equivalence: a bit is 1 where both are the same.

Example:

```
PRINT -1 EQV 0
```

Shows:

```
 0
```

See also: XOR

ERASE

Statement · `ERASE array[, array...]`

Deletes arrays, freeing their memory, so they can be dimensioned again.

Example:

```
DIM A(100): ERASE A: DIM A(5)
```

See also: DIM

ERDEV

Function · `ERDEV | ERDEV$`

GW-BASIC's device error details: always 0 and "" here.

See also: ERR

ERL

Function · `ERL`

The line number where the last error happened (0 in direct mode).

Example:

```
100 PRINT "Error"; ERR; "in"; ERL: RESUME NEXT
```

See also: ERR, ON ERROR GOTO

ERR

Function · `ERR`

The code of the last error (see Error codes).

Example:

```
10 ON ERROR GOTO 100
20 X = 1 / 0
30 END
100 PRINT ERR: RESUME NEXT
RUN
```

Shows:

```
 11
```

See also: ERL, ERROR

ERROR

Statement · `ERROR n`

Raises error `n` (1-255), as if it had happened: a test, or a program's own error codes.

- Codes without a message print `Unprintable error`.

Example:

```
ERROR 53
```

Shows:

```
File not found
```

See also: ON ERROR GOTO, ERR

EXIT FOR

Statement · `EXIT FOR`

Leaves the innermost FOR loop at once, going on after its NEXT (from QuickBASIC).

Example:

```
10 FOR I = 1 TO 10: IF I = 3 THEN EXIT FOR
20 NEXT: PRINT I
RUN
```

Shows:

```
 3
```

See also: FOR

EXP

Function · `EXP(x)`

e (2.718...) to the power `x`.

- Errors: `Overflow` (6) if too big.

Example:

```
PRINT EXP(1)
```

Shows:

```
 2.71828183
```

See also: LOG

EXTERR

Function · `EXTERR(n)`

DOS extended error information: always 0 here.

See also: ERR

FIELD

Statement · `FIELD #n, width AS var$[, width AS var$...]`

Divides random file `n`'s record buffer into string fields (at most 16, no more than the record length). GET fills them; LSET/RSET set them for PUT.

- Errors: `FIELD overflow` (50).

Example:

```
FIELD #1, 20 AS NM$, 2 AS AGE$
```

See also: GET, PUT, LSET

FILES

Command · `FILES [pattern$]`

Lists files, four to a line (folders marked `<DIR>`), then the free space on that drive.

- The pattern may have `*` and `?` and a drive and folder in front: `FILES "*.BAS"`, `FILES "SD:"`, `FILES "SD:/GAMES/*.BAS"`.
- Without one, all files in the current folder.
- `File not found` is printed (not an error) when nothing matches.

Example:

```
FILES "*.BAS"
```

Shows:

```
PRIMES.BAS          NEODEMO.BAS
3084288 Bytes free
```

See also: DIR$, KILL, CHDIR

FIX

Function · `FIX(x)`

`x` with its fraction cut off (towards 0).

Example:

```
PRINT FIX(-2.7); FIX(2.7)
```

Shows:

```
-2  2
```

See also: INT, CINT

FOR

Statement · `FOR v = a TO b [STEP s] ... NEXT [v]`

Repeats the lines up to NEXT with `v` = a, a+s, ... while it hasn't passed `b`. STEP defaults to 1; a negative STEP counts down.

- The loop body is skipped when `a` is already past `b`.
- Up to 32 loops nested.

Example:

```
FOR I = 10 TO 1 STEP -3: PRINT I;: NEXT
```

Shows:

```
 10  7  4  1
```

See also: NEXT, EXIT FOR, WHILE

FORMAT

Command · `FORMAT ON | FORMAT OFF | FORMAT n`

Whether LIST, EDIT and SAVE indent the program: FOR ... NEXT and WHILE ... WEND bodies by `n` spaces a level (2 to start with, 1-8), and in a program without line numbers the subroutines (the lines under a label GOSUB calls or ON ERROR GOTO goes to; labels stay at the left).

- Only the look changes: loading a program takes the indent off again.
- On at start-up.

Example:

```
10 FOR I = 1 TO 2
20 PRINT I
30 NEXT
LIST
```

Shows:

```
10 FOR I = 1 TO 2
20   PRINT I
30 NEXT
```

See also: LIST, EDIT, SAVE

FRE

Function · `FRE(x)`

The memory free for programs and data, in bytes (`x` is ignored).

Example:

```
PRINT FRE(0)
```

Shows:

```
 431456
```

FREEFILE

Function · `FREEFILE`

The lowest file number (1-4) not in use.

Example:

```
F = FREEFILE: OPEN "LOG.TXT" FOR APPEND AS #F
```

See also: OPEN

GET

Statement · `GET [#]n[, record]`

Reads record `record` (from 1; default the next) of random file `n` into its buffer and FIELD variables.

- Errors: `Bad record number` (63).

Example:

```
GET #1, 3: PRINT NM$
```

See also: PUT, FIELD

GOSUB

Statement · `GOSUB line | GOSUB label`

Calls the subroutine at `line` (or `label`); RETURN comes back to the statement after the GOSUB.

- Up to 32 deep.
- Errors: `Undefined line number` (8), `Undefined label` (80).

Example:

```
10 GOSUB 100: END
100 PRINT "SUB": RETURN
RUN
```

Shows:

```
SUB
```

See also: RETURN, ON...GOSUB

GOTO

Statement · `GOTO line | GOTO label`

Goes on at `line`, or at the line that starts with `label:` (a name and a colon).

- Labels: letters, digits, `.` and `_`, up to 40, any case; a program can have them with line numbers or without.
- Errors: `Undefined line number` (8), `Undefined label` (80), `Duplicate label` (81, at RUN).

Example:

```
10 PRINT "HI": GOTO 10
RUN
```

Shows:

```
HI
HI
...
```

See also: GOSUB, IF

HEX$

Function · `HEX$(n)`

`n` (-32768 to 65535) in hexadecimal; negative numbers as their 16-bit two's complement.

- Errors: `Overflow` (6) out of range.

Example:

```
PRINT HEX$(255); " "; HEX$(-1)
```

Shows:

```
FF FFFF
```

See also: OCT$, VAL

I2C

Statement · `I2C OPEN [sda, scl[, khz]] | I2C WRITE addr, item[, item...][, NOSTOP] | I2C SCAN | I2C CLOSE`

The chip's I2C. OPEN starts it (default GP2 SDA, GP3 SCL, I2C1, 100 kHz; 10-1000), WRITE sends bytes (numbers 0-255 and strings) to device `addr` (0-127) (NOSTOP keeps the bus for a read), SCAN lists the devices that answer, CLOSE frees the pins.

- SDA on an even pin, SCL the next one.
- Errors: `Device I/O error` (57) if no device answers, `Device unavailable` (68) before OPEN.

Example:

```
I2C OPEN: I2C SCAN
```

See also: I2CREAD, SPI

I2CREAD

Function · `I2CREAD(addr[, reg])`

Reads one byte from I2C device `addr`; with `reg`, first writes the register number.

Example:

```
PRINT I2CREAD(&H68, &H75)
```

Shows:

```
 104
```

See also: I2CREAD$, I2C

I2CREAD$

Function · `I2CREAD$(addr, n[, reg])`

Reads `n` bytes (1-255) from I2C device `addr` as a string; with `reg`, from that register.

Example:

```
D$ = I2CREAD$(&H68, 6, &H3B)
```

See also: I2CREAD

IF

Statement · `IF cond THEN statements [ELSE statements] | IF cond THEN line | IF cond GOTO line`

Runs the statements after THEN when `cond` is not 0 (true), else those after ELSE. A line number or label after THEN or ELSE jumps there (a label when nothing else follows it).

- Comparisons give -1 (true) or 0 (false); any number not 0 counts as true.
- Everything to the end of the line belongs to THEN or ELSE.

Example:

```
A = 5: IF A > 3 THEN PRINT "BIG" ELSE PRINT "SMALL"
```

Shows:

```
BIG
```

See also: WHILE, ON...GOTO

IMP

Operator · `a IMP b`

Bitwise implication: false only where `a` is true and `b` false.

Example:

```
PRINT 0 IMP 0
```

Shows:

```
-1
```

See also: EQV

INKEY$

Function · `INKEY$`

The key waiting, or "" if none: it doesn't wait. Cursor and function keys give CHR$(0) and the PC scan code (see INKEY$ key codes).

Example:

```
K$ = INKEY$: IF K$ = CHR$(0) + CHR$(72) THEN PRINT "UP"
```

See also: INPUT$, ON KEY

INPUT

Statement · `INPUT ["prompt"{; | ,}] var[, var...]`

Waits for a line typed at the terminal and puts its comma-separated items into the variables. A `;` after the prompt adds `?`, a `,` doesn't.

- If the items don't fit (too few, or text for a number) it says `?Redo from start` and asks again.
- Ctrl-C stops the program.

Example:

```
INPUT "Age"; A: PRINT A * 12; "months"
```

Shows:

```
Age? 3
 36 months
```

See also: LINE INPUT, INPUT #

INPUT

Statement · `INPUT #n, var[, var...]`

Reads items from file `n`: quoted strings, or text up to a comma or the end of the line (numbers stop at a space too).

- Errors: `Input past end` (62).

Example:

```
INPUT #1, N$, S
```

See also: WRITE #, LINE INPUT #

INPUT$

Function · `INPUT$(n[, [#]f])`

Reads `n` characters: from file `f`, or from the keyboard (waiting, without showing them).

Example:

```
K$ = INPUT$(1): PRINT "You pressed "; K$
```

Shows:

```
You pressed A
```

See also: INKEY$, INPUT

INSTR

Function · `INSTR([start,] s$, find$)`

Where `find$` first appears in `s$` (from position `start`, default 1); 0 if it doesn't.

Example:

```
PRINT INSTR("HELLO", "L"); INSTR(4, "HELLO", "L")
```

Shows:

```
 3  4
```

See also: MID$

INT

Function · `INT(x)`

The largest whole number not more than `x` (rounds down).

Example:

```
PRINT INT(-2.5); INT(2.5)
```

Shows:

```
-3  2
```

See also: FIX, CINT

KEY

Statement · `KEY n, text$ | KEY LIST | KEY ON | KEY OFF`

Sets what function key F1-F10 types (`CHR$(13)` in it is Enter), lists them, or shows their labels on row 25 (`KEY ON`) as on a PC.

- Defaults: F1 `LIST`, F2 `RUN`+Enter, F3 `LOAD"`, F4 `SAVE"`, F5 `CONT`+Enter, F7 `TRON`, F8 `TROFF`, F9 `KEY`, F10 `SCREEN 0,0,0`.
- `KEY(n) ON/OFF/STOP` is for key events: see ON KEY.

Example:

```
KEY 5, "FILES" + CHR$(13)
```

See also: ON KEY

KEY(n)

Statement · `KEY(n) ON | OFF | STOP`

Turns key `n`'s event (ON KEY) on, off, or holds it.

Example:

```
KEY(1) ON
```

See also: ON KEY

KILL

Statement · `KILL file$`

Deletes a file. No extension is added.

- Errors: `File not found` (53); `File already open` (55) while it is open.

Example:

```
KILL "OLD.BAS"
```

See also: NAME, FILES

LCASE$

Function · `LCASE$(s$)`

`s$` in lower case.

Example:

```
PRINT LCASE$("Hello")
```

Shows:

```
hello
```

See also: UCASE$

LEFT$

Function · `LEFT$(s$, n)`

The first `n` characters of `s$`.

Example:

```
PRINT LEFT$("HELLO", 2)
```

Shows:

```
HE
```

See also: RIGHT$, MID$

LEN

Function · `LEN(s$)`

The number of characters in `s$` (0-255).

Example:

```
PRINT LEN("HELLO")
```

Shows:

```
 5
```

LET

Statement · `[LET] var = expr`

Gives a variable a value. `LET` itself is optional.

- Errors: `Type mismatch` (13) for a string into a number or the other way; `Overflow` (6) out of an integer's range.

Example:

```
LET A = 5: B$ = "X"
```

See also: SWAP

LINE INPUT

Statement · `LINE INPUT ["prompt";] var$`

Reads a whole line typed at the terminal, commas and quotes included, into a string.

Example:

```
LINE INPUT "Name: "; N$
```

Shows:

```
Name: Ada Lovelace
```

See also: INPUT

LINE INPUT

Statement · `LINE INPUT #n, var$`

Reads a whole line of file `n` into a string.

Example:

```
LINE INPUT #1, L$
```

See also: INPUT #

LIST

Command · `LIST [range]`

Lists the program, pausing after each 28 lines (any key goes on, Q stops). `range` is `n`, `n-`, `-n` or `n-m`.

Example:

```
LIST 10-30
```

Shows:

```
10 PRINT "HELLO"
20 GOTO 10
```

See also: LLIST, EDIT

LLIST

Command · `LLIST [range]`

The same as LIST: there is no printer.

See also: LIST

LOAD

Command · `LOAD file$[, R]`

Loads a program, replacing the one in memory; with `,R` runs it too.

- `.BAS` is added if the name has no extension.
- Plain text (CR LF or LF lines) and GW-BASIC tokenised files are read.
- Errors: `File not found` (53).

Example:

```
LOAD "PRIMES": RUN
```

Shows:

```
 168 primes below 1000
```

See also: SAVE, RUN, MERGE

LOC

Function · `LOC(n)`

For a random file: the last record read or written. For COM1: the bytes waiting. Otherwise the position in 128-byte blocks.

See also: LOF, EOF

LOCATE

Statement · `LOCATE [row][, [col][, cursor]]`

Moves the cursor to `row` (1-30) and `col` (1-80); `cursor` 0 hides it, 1 shows it.

- Errors: `Illegal function call` (5) off the screen.

Example:

```
LOCATE 12, 35: PRINT "MIDDLE"
```

See also: CSRLIN, POS

LOF

Function · `LOF(n)`

The length of file `n` in bytes (for COM1: the room left in its buffer).

Example:

```
OPEN "PRIMES.BAS" FOR INPUT AS #1: PRINT LOF(1): CLOSE
```

Shows:

```
 329
```

See also: LOC

LOG

Function · `LOG(x)`

The natural logarithm of `x` (> 0). LOG(x) / LOG(10) is the base-10 log.

- Errors: `Illegal function call` (5) for 0 or less.

Example:

```
PRINT LOG(100) / LOG(10)
```

Shows:

```
 2
```

See also: EXP

LPRINT

Statement · `LPRINT items | LPRINT USING fmt$; items`

The same as PRINT: there is no printer.

See also: PRINT

LSET

Statement · `LSET var$ = s$`

Puts `s$` in a FIELD variable, left-aligned and padded with spaces (cut if too long). Use it, not LET, for field variables.

Example:

```
LSET NM$ = "Grace"
```

See also: RSET, FIELD

MERGE

Command · `MERGE file$`

Adds the lines of a plain-text program to the one in memory; lines with the same number are replaced.

Example:

```
MERGE "SUBS"
```

See also: LOAD, CHAIN

MID$

Function · `MID$(s$, start[, n])`

Up to `n` characters of `s$` from position `start` (all the rest without `n`).

- Errors: `Illegal function call` (5) for `start` < 1.

Example:

```
PRINT MID$("HELLO", 2, 3)
```

Shows:

```
ELL
```

See also: LEFT$, RIGHT$, MID$ =

MID$ =

Statement · `MID$(v$, start[, n]) = x$`

Replaces characters of `v$` in place, from `start`, with those of `x$` (at most `n`); `v$` keeps its length.

Example:

```
A$ = "HELLO": MID$(A$, 2, 3) = "ipp": PRINT A$
```

Shows:

```
HippO
```

See also: MID$

MKD$

Function · `MKD$(x)`

The number `x` as an 8-byte double-precision string, for random files.

See also: CVD

MKDIR

Statement · `MKDIR path$`

Makes a folder.

- Errors: `Path/File Access Error` (75) if it exists.

Example:

```
MKDIR "SD:/GAMES"
```

See also: CHDIR, RMDIR

MKI$

Function · `MKI$(n)`

The integer `n` (-32768 to 32767) as a 2-byte string, low byte first.

- Errors: `Overflow` (6).

Example:

```
PRINT LEN(MKI$(5))
```

Shows:

```
 2
```

See also: CVI

MKS$

Function · `MKS$(x)`

The number `x` as a 4-byte single-precision string (GW-BASIC's format).

See also: CVS

NAME

Statement · `NAME old$ AS new$`

Renames a file (or moves it to another folder on the same drive).

- Errors: `File not found` (53), `File already exists` (58).

Example:

```
NAME "A.BAS" AS "B.BAS"
```

See also: KILL

NEOPIXEL

Statement · `NEOPIXEL OPEN [pin[, count[, order$]]] | NEOPIXEL n, r, g, b | NEOPIXEL FILL r, g, b | NEOPIXEL SHOW | NEOPIXEL BRIGHT level | NEOPIXEL CLEAR | NEOPIXEL CLOSE`

WS2812B LEDs ("NeoPixels") on one pin, through a PIO state machine. OPEN sets up `count` (1-1024, default 1) LEDs on `pin` (default GP28, the Maker Pi Pico's RGB LED), GRB order or "RGB"; `n, r, g, b` and FILL set colours (0-255) in the buffer; SHOW sends it; BRIGHT (0-255) scales the next SHOWs; CLEAR turns them all off; CLOSE frees the pin.

- Errors: `Device unavailable` (68) before OPEN, `Illegal function call` (5) for a value out of range.
- RUN, NEW and SYSTEM close it.

Example:

```
NEOPIXEL OPEN: NEOPIXEL BRIGHT 40: NEOPIXEL 0, 255, 0, 0: NEOPIXEL SHOW
```

See also: PIN, PWM

NEW

Command · `NEW`

Deletes the program and its variables, closes files, and frees the pins BASIC used.

See also: CLEAR, DELETE

NEXT

Statement · `NEXT [v[, v...]]`

Ends a FOR loop: adds the step and goes round again unless the end is passed. `NEXT J, I` ends two loops.

- Errors: `NEXT without FOR` (1).

Example:

```
FOR I = 1 TO 3: PRINT I;: NEXT I
```

Shows:

```
 1  2  3
```

See also: FOR

NOT

Operator · `NOT a`

Bitwise NOT: -1 becomes 0 and 0 becomes -1 (NOT x is -x - 1).

Example:

```
PRINT NOT 0; NOT 5
```

Shows:

```
-1 -6
```

See also: AND

NUMBERS

Command · `NUMBERS OFF | NUMBERS ON`

`NUMBERS OFF` takes the line numbers off the program: each line a GOTO, GOSUB, THEN, ELSE, RESTORE, RESUME or RUN goes to gets a label, L and its old number (`L100:`), and those use the label. `NUMBERS ON` numbers the lines 10, 20, 30... again, keeping the labels.

- A program without numbers is listed, saved and edited without them; its lines' numbers (in errors, ERL, TRON) are their lines in the file.
- Numbers compared with ERL (`IF ERL = 100`) aren't changed.

Example:

```
10 PRINT "HI": GOTO 10
NUMBERS OFF
LIST
```

Shows:

```
L10: PRINT "HI": GOTO L10
```

See also: RENUM, GOTO, EDIT

OCT$

Function · `OCT$(n)`

`n` (-32768 to 65535) in octal.

Example:

```
PRINT OCT$(8)
```

Shows:

```
10
```

See also: HEX$

ON COM

Statement · `ON COM(1) GOSUB line`

Calls the subroutine when bytes arrive on the open COM1 (with COM(1) ON).

See also: COM(1), OPEN

ON ERROR GOTO

Statement · `ON ERROR GOTO line | ON ERROR GOTO label | ON ERROR GOTO 0`

Sends errors in a running program to an error handler at `line`. `ON ERROR GOTO 0` turns that off; inside a handler it reports the error and stops.

- ERR and ERL say what and where; RESUME ends the handler.
- Ctrl-C can't be trapped.
- A program that ends inside a handler stops with `No RESUME` (19).

Example:

```
10 ON ERROR GOTO 100
20 OPEN "NONE" FOR INPUT AS #1
30 END
100 PRINT "no file": RESUME 30
RUN
```

Shows:

```
no file
```

See also: RESUME, ERR, ERROR

ON KEY

Statement · `ON KEY(n) GOSUB line`

Calls the subroutine when key `n` is pressed (with KEY(n) ON): 1-10 are F1-F10, 11 Up, 12 Left, 13 Right, 14 Down.

- `KEY 15`-`KEY 20` user keys are accepted but never trigger.

Example:

```
10 ON KEY(1) GOSUB 100: KEY(1) ON
20 GOTO 20
100 PRINT "F1": RETURN
RUN
```

Shows:

```
F1
```

See also: KEY(n), INKEY$

ON PLAY

Statement · `ON PLAY(n) GOSUB line`

Calls the subroutine when PLAY's background queue drops below `n` notes (with PLAY ON): for music that never stops.

See also: PLAY, PLAY ON

ON TIMER

Statement · `ON TIMER(s) GOSUB line`

Calls the subroutine every `s` seconds (1-86400), with TIMER ON.

Example:

```
10 ON TIMER(1) GOSUB 100: TIMER ON
20 GOTO 20
100 PRINT TIME$: RETURN
RUN
```

Shows:

```
12:00:01
12:00:02
```

See also: TIMER ON

ON...GOSUB

Statement · `ON n GOSUB line1[, line2...] (lines or labels)`

Calls the n-th subroutine in the list. If `n` is 0 or past the list, it goes on with the next statement.

Example:

```
10 ON 2 GOSUB 100, 200: END
100 PRINT "A": RETURN
200 PRINT "B": RETURN
RUN
```

Shows:

```
B
```

See also: ON...GOTO, GOSUB

ON...GOTO

Statement · `ON n GOTO line1[, line2...] (lines or labels)`

Goes to the n-th line in the list; with `n` 0 or past the list, goes on.

Example:

```
ON 1 GOTO 100, 200
```

See also: ON...GOSUB

OPEN

Statement · `OPEN file$ FOR INPUT | OUTPUT | APPEND | RANDOM AS [#]n [LEN = size] | OPEN mode$, [#]n, file$[, size] | OPEN "COM1:..." AS [#]n`

Opens a file as number `n` (1-4). OUTPUT creates or empties it, APPEND adds to its end, INPUT reads it, RANDOM reads and writes fixed-size records (LEN 1-1024, default 128; also the default without FOR).

- The short form takes "I", "O", "A" or "R".
- A "COM1:" name opens the serial port: see the manual's COM1 section.
- Errors: `File not found` (53), `File already open` (55), `Bad file number` (52).

Example:

```
OPEN "SCORES.TXT" FOR OUTPUT AS #1: PRINT #1, "Ann", 120: CLOSE #1
```

See also: CLOSE, PRINT #, INPUT #

OPTION BASE

Statement · `OPTION BASE 0 | 1`

The lowest array subscript, before any array is used.

Example:

```
OPTION BASE 1: DIM A(3)
```

See also: DIM

OR

Operator · `a OR b`

Bitwise OR: true if either is.

Example:

```
PRINT 12 OR 3
```

Shows:

```
 15
```

See also: AND, XOR

PAUSE

Statement · `PAUSE ms`

Waits `ms` milliseconds (an extension); Ctrl-C still stops it.

Example:

```
PAUSE 500
```

See also: TIMER

PEEK

Function · `PEEK(offset)`

A byte of the emulated memory chosen by DEF SEG: the text screen (&HB800: reads spaces and colour 7, as a terminal can't be read) or BIOS data (DEF SEG = 0: &H449 mode 3, &H44A columns 80, &H484 rows-1 29, &H46C-&H46E timer ticks).

- Errors: `Illegal function call` (5) anywhere else.

Example:

```
DEF SEG = 0: PRINT PEEK(&H44A)
```

Shows:

```
 80
```

See also: POKE, DEF SEG

PIN

Statement / function · `PIN pin, level | PIN(pin)`

`PIN pin, level` drives a pin high (1) or low (0), making it an output first. `PIN(pin)` reads it (0 or 1), making an unused pin an input. `LED` stands for the board's LED (GP25).

- Pins BASIC uses (the speaker, SD card, PSRAM, open I2C/SPI/COM1/NeoPixel) are refused: `GPn is used by ...` then `Illegal function call`.
- On the Maker Pi Pico the buttons GP20-22 read 0 when pressed (turn their pull-ups on with PINMODE).

Example:

```
PIN LED, 1: PRINT PIN(20)
```

Shows:

```
 1
```

See also: PINMODE, PINS, PWM

PINMODE

Statement · `PINMODE pin, INPUT | OUTPUT | PULLUP | PULLDOWN`

Sets a pin up as an input, an output, or an input with its pull-up or pull-down (or modes 0-3).

- On the RP2350 a pull-down may not pull a pin low (its erratum E9); pull-ups are fine.

Example:

```
PINMODE 20, PULLUP: PRINT PIN(20)
```

Shows:

```
 1
```

See also: PIN

PINS

Command · `PINS`

Lists every pin and what it is doing (free, in, out, PWM, ADC, the speaker, SD card, I2C, SPI, COM1, NeoPixel...).

Example:

```
PINS
```

Shows:

```
GP0  free          GP8  free  ...
```

See also: PIN

PLAY

Statement · `PLAY music$`

Plays notes in GW-BASIC's music language (see the manual's Sound section: A-G, O, L, T, P, MF/MB...).

- After `MB` the notes play in the background (up to 32 queued) while the program goes on.
- `X` substrings aren't supported; `=variable;` is.

Example:

```
PLAY "T160 O3 L8 CDEFGAB > C"
```

See also: SOUND, PLAY(n), ON PLAY

PLAY ON

Statement · `PLAY ON | OFF | STOP`

Turns the PLAY event (ON PLAY) on, off, or holds it.

See also: ON PLAY

PLAY(n)

Function · `PLAY(n)`

How much is still waiting in the background queue (`n` is ignored): each note counts twice, as the note and the short gap after it (MN, normal style).

Example:

```
PLAY "MB CDE": PRINT PLAY(0)
```

Shows:

```
 6
```

See also: PLAY, ON PLAY

POKE

Statement · `POKE offset, byte`

Writes a byte to the emulated memory (DEF SEG). The text screen and BIOS bytes accept it and change nothing here.

See also: PEEK

POS

Function · `POS(n)`

The column the cursor is in, 1-80 (`n` is ignored).

Example:

```
PRINT "ABC";: PRINT POS(0)
```

Shows:

```
ABC 4
```

See also: CSRLIN

PRINT

Statement · `PRINT [items]`

Prints numbers and strings. `;` between items prints them together, `,` jumps to the next 14-column zone; a `;` or `,` at the end keeps the cursor on the line.

- Numbers print with a space before them (or a `-`) and one after.
- `TAB(n)` and `SPC(n)` position the text.
- Lines wrap at 80 columns.

Example:

```
PRINT "A"; 1, "B"; 2
```

Shows:

```
A 1          B 2
```

See also: PRINT USING, WRITE, TAB

PRINT

Statement · `PRINT #n, [USING fmt$;] items`

Like PRINT, into file `n`. Lines end in CR LF (CR alone on COM1 unless LF was asked for), and there is no 80-column wrap.

Example:

```
PRINT #1, "TOTAL"; 42
```

See also: WRITE #, PRINT

PRINT USING

Statement · `PRINT USING fmt$; items[;]`

Prints numbers and strings in the fields of `fmt$` (see PRINT USING formats); the format is used again from its start for more items.

- Also `PRINT #n, USING`, and after other items: `PRINT "X ="; USING "##.#"; X`.
- A number too big for its field is printed in full after a `%`.

Example:

```
PRINT USING "###.##"; 3.14159; -2
```

Shows:

```
  3.14 -2.00
```

See also: PRINT

PUT

Statement · `PUT [#]n[, record]`

Writes random file `n`'s buffer as record `record` (from 1; default the one after the last).

Example:

```
LSET NM$ = "Ada": PUT #1, 1
```

See also: GET, FIELD

PWM

Statement · `PWM pin, freq[, duty]`

Runs PWM on a pin at `freq` (8 Hz-10 MHz) with a duty of 0-100 % (default 50); `PWM pin, 0` stops it.

- Pins in pairs (GP0/1, GP2/3...) share one frequency, and GP16-31 share GP0-15's slices: changing one warns about its partner.

Example:

```
PWM LED, 1000, 10
```

See also: SERVO, PIN

RANDOMIZE

Statement · `RANDOMIZE [seed]`

Starts RND's sequence afresh: from the clock without a seed, else from `seed` (the same seed, the same numbers).

Example:

```
RANDOMIZE: PRINT INT(RND * 6) + 1
```

Shows:

```
 4
```

See also: RND

READ

Statement · `READ var[, var...]`

Takes the next items from DATA lines, in order.

- Errors: `Out of DATA` (4); `Syntax error` reading a string into a number.

Example:

```
10 READ A, B$: PRINT A; B$
20 DATA 5, CATS
RUN
```

Shows:

```
 5 CATS
```

See also: DATA, RESTORE

REM

Statement · `REM text | ' text`

A remark: the rest of the line is ignored. `'` is short for `:REM`.

Example:

```
10 REM Setup
20 X = 1 ' start
```

RENUM

Command · `RENUM [new][, [old][, step]]`

Renumbers program lines from line `old` (default the first) as `new`, `new+step`... (default 10 and 10), and changes the line numbers after GOTO, GOSUB, THEN, ELSE, RESTORE, RESUME and RUN to match.

- Not for a program without line numbers (NUMBERS ON first).

Example:

```
RENUM 100, , 10
```

See also: AUTO, DELETE

RESET

Statement · `RESET`

Closes every open file.

See also: CLOSE

RESTORE

Statement · `RESTORE [line | label]`

Makes READ start again from the first DATA (or from `line` or `label`).

Example:

```
RESTORE 500
```

See also: READ, DATA

RESUME

Statement · `RESUME | RESUME NEXT | RESUME line | RESUME label`

Ends an error handler: `RESUME` retries the statement that failed, `RESUME NEXT` goes on after it, `RESUME line` goes to the line.

- Errors: `RESUME without error` (20).

Example:

```
100 PRINT "skipped": RESUME NEXT
```

See also: ON ERROR GOTO

RETURN

Statement · `RETURN`

Returns from a GOSUB (or an event subroutine) to the statement after the call. It also ends FOR and WHILE loops started in the subroutine.

- Errors: `RETURN without GOSUB` (3).

See also: GOSUB

RIGHT$

Function · `RIGHT$(s$, n)`

The last `n` characters of `s$`.

Example:

```
PRINT RIGHT$("HELLO", 3)
```

Shows:

```
LLO
```

See also: LEFT$

RMDIR

Statement · `RMDIR path$`

Removes an empty folder.

- Errors: `Path/File Access Error` (75) if it isn't empty.

Example:

```
RMDIR "GAMES"
```

See also: MKDIR

RND

Function · `RND | RND(x)`

A random number from 0 to just under 1. `RND(0)` repeats the last one; a negative `x` starts a sequence from it.

- RUN always starts the same sequence: RANDOMIZE for a different one.

Example:

```
PRINT INT(RND * 6) + 1
```

Shows:

```
 5
```

See also: RANDOMIZE

RSET

Statement · `RSET var$ = s$`

Like LSET, right-aligned.

Example:

```
RSET N$ = STR$(42)
```

See also: LSET

RUN

Command · `RUN [line | label] | RUN file$`

Runs the program from the start (or from `line`), after clearing the variables, closing files and resetting DATA; `RUN file$` loads the file first.

- Pins used by the last run are set back to inputs.

Example:

```
RUN "PRIMES"
```

Shows:

```
 168 primes below 1000
```

See also: LOAD, CONT

SAVE

Command · `SAVE file$[, A]`

Saves the program as plain text. `.BAS` is added if the name has no extension; `,A` is accepted (text is the only format).

- `SAVE "SD:GAME"` saves to the SD card; `SAVE "AUTORUN"` makes it run at power-up.

Example:

```
SAVE "HELLO"
```

See also: LOAD, FILES

SCREEN(r, c)

Function · `SCREEN(row, col[, 1])`

The character at a screen position, or with 1 its colours. A terminal can't be read back, so it gives 32 (a space), or 7.

Example:

```
PRINT SCREEN(1, 1)
```

Shows:

```
 32
```

See also: PEEK

SERVO

Statement · `SERVO pin, angle`

Drives a hobby servo to `angle` 0-180 degrees (0.5-2.5 ms pulses at 50 Hz).

Example:

```
SERVO 16, 90
```

See also: PWM

SGN

Function · `SGN(x)`

1 if `x` is positive, -1 if negative, 0 if 0.

Example:

```
PRINT SGN(-7); SGN(0); SGN(3)
```

Shows:

```
-1  0  1
```

See also: ABS

SIN

Function · `SIN(x)`

The sine of `x` radians.

Example:

```
PRINT SIN(3.14159 / 2)
```

Shows:

```
 1
```

See also: COS, TAN, ATN

SOUND

Statement · `SOUND freq, ticks`

A tone of `freq` Hz (37-32767; 32767 is silence) for `ticks` (18.2 make a second). After PLAY "MB" it joins the background queue; ticks 0 stops the sound.

- Errors: `Illegal function call` (5) out of range.

Example:

```
SOUND 440, 18
```

See also: BEEP, PLAY

SPACE$

Function · `SPACE$(n)`

A string of `n` (0-255) spaces.

Example:

```
PRINT "["; SPACE$(3); "]"
```

Shows:

```
[   ]
```

See also: STRING$, SPC

SPC

Function · `SPC(n)`

In PRINT, prints `n` spaces.

Example:

```
PRINT "A"; SPC(3); "B"
```

Shows:

```
A   B
```

See also: TAB, SPACE$

SPI

Statement · `SPI OPEN [sck, mosi, miso[, khz[, mode]]] | SPI WRITE item[, item...] | SPI CLOSE`

The chip's SPI. OPEN starts it (default SPI0: GP6 SCK, GP7 MOSI, GP4 MISO; 1000 kHz, 1-50000; mode 0-3), WRITE sends bytes ignoring what comes back, CLOSE frees the pins. Drive chip select yourself with PIN (GP5 is free for it).

- SPI1 (GP10-15) is the SD card's while a card is in.
- Errors: `Device unavailable` (68) before OPEN.

Example:

```
PIN 5, 1: SPI OPEN: PIN 5, 0: SPI WRITE &H9F: ID$ = SPI$(STRING$(3, 0)): PIN 5, 1
```

See also: SPI$, I2C

SPI$

Function · `SPI$(out$)`

Sends the bytes of `out$` on the open SPI and returns the bytes that came back meanwhile (as many).

Example:

```
R$ = SPI$(STRING$(4, 0))
```

See also: SPI

SQR

Function · `SQR(x)`

The square root of `x` (0 or more).

- Errors: `Illegal function call` (5) for a negative `x`.

Example:

```
PRINT SQR(2)
```

Shows:

```
 1.41421356
```

STOP

Statement · `STOP`

Stops the program with `Break in <line>`; CONT carries on. Files stay open.

See also: END, CONT

STR$

Function · `STR$(x)`

The number `x` as PRINT shows it, with a space before a positive number.

Example:

```
PRINT "<"; STR$(42); ">"
```

Shows:

```
< 42>
```

See also: VAL

STRING$

Function · `STRING$(n, c | c$)`

`n` (0-255) copies of character code `c` or of the first character of `c$`.

Example:

```
PRINT STRING$(5, "*"); STRING$(3, 65)
```

Shows:

```
*****AAA
```

See also: SPACE$

SWAP

Statement · `SWAP a, b`

Exchanges two variables of the same type.

- Errors: `Type mismatch` (13).

Example:

```
A = 1: B = 2: SWAP A, B: PRINT A; B
```

Shows:

```
 2  1
```

See also: LET

SYSTEM

Command · `SYSTEM`

Leaves BASIC and starts it afresh, as after a reset (the files on the disks stay; AUTORUN.BAS isn't run again).

See also: NEW

TAB

Function · `TAB(n)`

In PRINT, moves to column `n` (1-80); to the next line first if the cursor is already past it.

- As in GW-BASIC: at column `n` already, it does nothing; past it, it goes to column `n` of the next line. So `PRINT T; TAB(4); "hello"` stays on one line for T = 5 (" 5 " fills columns 1-3) but not for T = 10 (" 10 " fills 1-4).
- Give TAB room for the widest number (`TAB(6)` for up to 999), or a fixed width: `PRINT USING "### "; T;: PRINT "hello"`.

Example:

```
PRINT "A"; TAB(10); "B"
```

Shows:

```
A         B
```

See also: SPC, LOCATE

TAN

Function · `TAN(x)`

The tangent of `x` radians.

Example:

```
PRINT TAN(.7854)
```

Shows:

```
 1.00000367
```

See also: ATN

TEMP

Function · `TEMP()`

The chip's temperature in °C (uncalibrated: within a few degrees).

Example:

```
PRINT INT(TEMP() * 10) / 10
```

Shows:

```
 27.3
```

See also: ADC

TIME$

Statement / function · `TIME$ = "hh:mm:ss" | TIME$`

Sets the time of day, or gives it as "hh:mm:ss" (24-hour, UTC).

Example:

```
TIME$ = "09:30:00": PRINT TIME$
```

Shows:

```
09:30:00
```

See also: DATE$, TIMER

TIMER

Function · `TIMER`

Seconds since the board started, with fractions: for timing things.

Example:

```
T = TIMER: PAUSE 500: PRINT TIMER - T
```

Shows:

```
 .500118999
```

See also: PAUSE, ON TIMER

TIMER ON

Statement · `TIMER ON | OFF | STOP`

Turns the timer event (ON TIMER) on, off, or holds it.

See also: ON TIMER

TROFF

Command · `TROFF`

Turns line tracing off.

See also: TRON

TRON

Command · `TRON`

Turns line tracing on: each line's number is printed in brackets as it runs.

Example:

```
TRON: RUN
```

Shows:

```
[10][20][10][20]...
```

See also: TROFF

UCASE$

Function · `UCASE$(s$)`

`s$` in upper case.

Example:

```
PRINT UCASE$("Hello")
```

Shows:

```
HELLO
```

See also: LCASE$

VAL

Function · `VAL(s$)`

The number at the start of `s$` (spaces skipped; `&H` hex and `&O` octal too); 0 if there isn't one.

Example:

```
PRINT VAL("12.5KG"); VAL("&HFF"); VAL("X")
```

Shows:

```
 12.5  255  0
```

See also: STR$

VIEW PRINT

Statement · `VIEW PRINT [top TO bottom]`

Makes only rows `top` to `bottom` scroll (an ANSI scroll region); `VIEW PRINT` alone undoes it.

Example:

```
VIEW PRINT 5 TO 20
```

See also: LOCATE, CLS

WEND

Statement · `WEND`

Ends a WHILE loop.

- Errors: `WEND without WHILE` (30).

See also: WHILE

WHILE

Statement · `WHILE cond ... WEND`

Repeats the lines up to WEND while `cond` is true (not 0); tested before each pass.

- Up to 32 nested.
- Errors: `WHILE without WEND` (29).

Example:

```
I = 1: WHILE I < 100: I = I * 2: WEND: PRINT I
```

Shows:

```
 128
```

See also: FOR, IF

WIDTH

Statement · `WIDTH n`

Accepted for old programs; the screen stays 80 columns.

WRITE

Statement · `WRITE [items]`

Prints the items separated by commas, strings in quotes, numbers without spaces: what INPUT # reads back exactly.

Example:

```
WRITE "Ann", 120
```

Shows:

```
"Ann",120
```

See also: WRITE #, PRINT

WRITE

Statement · `WRITE #n, items`

Writes the items to file `n` separated by commas, strings in quotes: INPUT # reads them back exactly.

Example:

```
WRITE #1, "Ann", 120
```

See also: INPUT #

XOR

Operator · `a XOR b`

Bitwise exclusive OR: true if just one is.

Example:

```
PRINT 12 XOR 10
```

Shows:

```
 6
```

See also: OR

XRECV

Statement · `XRECV file$`

Receives a file from the terminal by XMODEM and saves it as `file$`, replacing one of that name. Start the terminal's XMODEM send within a minute.

- CRC-16 or checksum; 128- or 1K-byte blocks. XMODEM's ^Z padding is taken off the end.
- A failed transfer deletes the partial file and gives `Device I/O error` (57). Ctrl-X a few times cancels.
- Works to either drive: `XRECV "SD:PIC.BMP"`.

Example:

```
XRECV "GAME.BAS"
```

Shows:

```
Start the XMODEM send on the terminal now.
Received 1 file, 2048 bytes.
```

See also: XSEND, LOAD

XSEND

Statement · `XSEND file$`

Sends a file to the terminal by XMODEM (1K blocks with CRC-16, or 128-byte ones with a checksum if the receiver asks).

- Errors: `Device I/O error` (57) if it fails or the file can't be read.

Example:

```
XSEND "GAME.BAS"
```

Shows:

```
Start the XMODEM receive on the terminal now.
Sent 1 file, 2048 bytes.
```

See also: XRECV

\ and MOD

Operator · `a \ b, a MOD b`

Integer division (both rounded to whole numbers first, the result cut towards 0) and its remainder.

Example:

```
PRINT 17 \ 5; 17 MOD 5
```

Shows:

```
 3  2
```

See also: + - * / ^

Appendix A: error codes

Code	Message	Code	Message
1	NEXT without FOR	26	FOR without NEXT
2	Syntax error	29	WHILE without WEND
3	RETURN without GOSUB	30	WEND without WHILE
4	Out of DATA	50	FIELD overflow
5	Illegal function call	52	Bad file number
6	Overflow	53	File not found
7	Out of memory	54	Bad file mode
8	Undefined line number	55	File already open
9	Subscript out of range	57	Device I/O error
10	Duplicate Definition	58	File already exists
11	Division by zero	61	Disk full
12	Illegal direct	62	Input past end
13	Type mismatch	63	Bad record number
15	String too long	64	Bad file name
17	Can't continue	67	Too many files
18	Undefined user function	68	Device unavailable
19	No RESUME	70	Permission denied
20	RESUME without error	71	Disk not ready
22	Missing operand	73	Feature not available
24	Device Timeout	75	Path/File Access Error
25	Device Fault	76	Path not found

(Other GW-BASIC codes, 14, 16, 23, 27, 51, 66, 69, 72 and 74, have their messages for ERROR n; unknown codes print `Unprintable error`.)

Appendix B: PRINT USING formats

Field	Meaning
`#`	a digit position
`.`	the decimal point
`,` before the point	commas between thousands
`+` first or last	always show the sign there
`-` last	a minus after negative numbers
`**`	fill the space before the number with `*`
`$$`	a `$` just before the number (`**$` does both)
`^^^^`	exponent form (`##.##^^^^` prints `2.35E+02`)
`!`	the first character of a string
`\ \`	the first n + 2 characters of a string
`&`	the whole string
`_`	prints the next character as it is

Appendix C: INKEY$ key codes

Ordinary keys give their character. These give CHR$(0) followed by the PC scan code:

Key	Code	Key	Code
Up	72	Home	71
Down	80	End	79
Left	75	PgUp	73
Right	77	PgDn	81
F1-F10	59-68	Ins	82
		Del	83

Esc gives CHR$(27), Enter CHR$(13), Backspace CHR$(8), Tab CHR$(9), Ctrl-A to Ctrl-Z CHR$(1)-CHR$(26) (Ctrl-C stops the program).

Appendix D: characters

BASIC uses the IBM PC character set (code page 437) and sends each character to the terminal as its Unicode picture in UTF-8: CHR$(1) ☺, CHR$(3) ♥, CHR$(176)-CHR$(178) ░▒▓, CHR$(179) │, CHR$(196) ─, CHR$(218) ┌, CHR$(219) █, CHR$(227) π, CHR$(248) °. Control codes print their pictures too, except 7 (beep), 8 (backspace), 9 (tab), 10 (line feed), 12 (clear screen), 13 (carriage return) and 27 (Esc, for ANSI codes); 11 moves the cursor home and 28-31 move it right, left, up and down.

Appendix E: pins BASIC uses

Pin	Used by	Boards
GP18	the speaker (BEEP, SOUND, PLAY)	all (the Maker Pi Pico's buzzer)
GP10-15	the SD card, while one is in	all (the Maker Pi Pico's slot)
GP47	the PSRAM	Pico Plus 2
GP28	NEOPIXEL's default (the Maker Pi Pico's RGB LED)	while open
GP20	held low at power-up: skip AUTORUN.BAS	all (the Maker Pi Pico's first button)
GP2/3, GP4-7, GP8/9	I2C, SPI and COM1's defaults	while open

The console is the board's USB port, so GP0 and GP1 are free. PINS lists what each pin is doing.